CRINÔMETRO v4.5.0

DOCUMENTAÇÃO TÉCNICA, TEÓRICA E METODOLÓGICA

Arquitetura Computacional, Processamento Digital de Sinais (DSP), Aprendizado Ativo e Metrologia Bioacústica de Alta Precisão para Sistemática e Descoberta Taxonômica em Orthoptera

Autor Técnico: Engenharia de Processamento Digital de Sinais, Arquitetura Python/Qt & Bioacústica Computacional
Classificação do Instrumento: Metrologia Pura para Diagnose e Caracterização Acústica de Novas Espécies
Padrão Terminológico Internacional: Ragge & Reynolds (1998), Bennet-Clark (1999), Márquez et al. (2001), Baker & Chesmore (2020)
Versão de Produção: v4.5.0 (Data de Emissão: 16/10/2026)

[DIRETRIZ DE DOMÍNIO CRÍTICA]

Axioma de Independência Taxonômica da Versão 4.5.0 (Proscrição da Lei de Dolbear):
O Crinômetro v4.5.0 é expressamente um instrumento de metrologia bioacústica de alta precisão voltado à caracterização, diagnose e descrição formal de **espécies novas e espécimes não documentados**. Nas versões anteriores (v4.3.2/v4.4.1), mantinha-se a regressão linear termo-acústica de Dolbear (1897), calibrada no século XIX unicamente para *Oecanthus fultoni* na América do Norte. **Na arquitetura v4.5.0, qualquer estimativa térmica indireta por equações empíricas fixas foi banida e erradicada.**

Justificativa Científica: A cinética do Gerador Central de Padrões (CPG) e os coeficientes térmicos (Q10) são caracteres espécie-específicos que variam com a massa torácica, fisiologia e microclima. Aplicar fórmulas pré-calibradas a táxons desconhecidos introduz viés sistemático inaceitável em taxonomia integrativa. O Crinômetro reporta exclusivamente grandezas físicas diretas: frequências em Hz, tempos em ms e densidades espectrais em dB/Hz.

SUMÁRIO ESTRUTURADO DA OBRA

1.0. CAPÍTULO 1: ARQUITETURA DE SOFTWARE, CONCORRÊNCIA E PIPELINE DE DADOS

- 1.1. Topologia de Fluxo de Dados (Dataflow) e Pipeline de Processamento Reativo
- 1.2. Refatoração Arquitetural: Transição Monolito -> MVC Desacoplado (SessionContext e Mediadores)
- 1.3. Concorrência Assíncrona: GenericWorker, Cancelamento Cooperativo e WorkerRegistry Monotônico
- 1.4. Persistência Transacional Atômica: Write-Ahead, Fsync e Prevenção de Corrupção em Disco
- Glossário Técnico e Científico da Seção 1 ([G-01] a [G-05])

2.0. CAPÍTULO 2: PROCESSAMENTO DIGITAL DE SINAIS (DSP) E ENGENHARIA ESPECTRAL

- 2.1. Condicionamento Primário: Seleção RMS Estéreo, Centralização DC e Filtro Anti-Vento (50 Hz)
- 2.2. Filtragem Passa-Faixa Butterworth de Fase Zero via Seções de Segunda Ordem (SOS)
- 2.3. Transformada de Fourier de Tempo Curto (STFT) e o Limite de Gabor-Heisenberg
- 2.4. Refinamento Parabólico Sub-Bin da Frequência Portadora (Método de Smith-Serra)
- 2.5. Densidade Espectral de Potência (Método de Welch) e Otimização Gráfica
- Glossário Técnico e Científico da Seção 2 ([G-06] a [G-12])

3.0. CAPÍTULO 3: MORFOLOGIA TEMPORAL, ENVOLTÓRIA ANALÍTICA E SINTAXE DE CHILREIOS

- 3.1. Biofísica Tegminal: Plectro, Lima Estridulatória e Dinâmica do Gerador Central de Padrões (CPG)
- 3.2. Envoltória Analítica via Transformada de Hilbert e Kernel de Suavização de Hann
- 3.3. Algoritmo de Detecção de Pulsos em 3 Fases e Metrologia Fina de Sílabas
- 3.4. Identificação da Frequência Portadora Modal e Filtragem de Rejeição de Banda Estreita
- 3.5. Agrupamento Hierárquico em 5 Camadas e Filtro de Coerência Rítmica Linear O(N)
- 3.6. Tratamento de Ruído e Diferenciação de Pausa Fisiológica vs. Ruído Transitório
- 3.7. Diagnóstico de Cadência Rítmica por Regressão Linear Ordinária e Inferência por Teste t
- 3.8. Tabela Formal das 15 Métricas Biométricas de Estridulação (Offset-to-Onset)
- Glossário Técnico e Científico da Seção 3 ([G-13] a [G-18])

4.0. CAPÍTULO 4: APRENDIZADO DE MÁQUINA HÍBRIDO E ENGENHARIA DE ATRIBUTOS

- 4.1. Acústica Ecológica: Segregação do Canto Focal em Coros Polifônicos e Invariância a Ganho
- 4.2. O Espaço Vetorial de 20 Descritores Bioacústicos Normalizados
- 4.3. Classificador Não-Supervisionado: Modelo de Mistura de Gaussianas (GMM Bimodal)
- 4.4. Classificador Supervisionado (HistGradientBoosting) e Fallback para RandomForest
- 4.5. Mineração de Negativos Difíceis (Hard Negative Mining) e Poda Contrastiva
- 4.6. Pontuação Híbrida Ponderada e Resgate Contextual (Contextual Gating)
- 4.7. Ciclo de Feedback do Aprendizado Ativo em 5 Passos (subpassos a até g)
- 4.8. Serialização Segura de Modelos via skops e Desacoplamento de Locks de Concorrência
- Glossário Técnico e Científico da Seção 4 ([G-19] a [G-24])

5.0. CAPÍTULO 5: COMPUTAÇÃO GRÁFICA, ACELERAÇÃO LOD E INTERFACE REATIVA

- 5.1. Teorema de Preservação de Transientes por Decimação Min-Max LoD
- 5.2. Arquitetura das Engines Vetorizadas: HighPerfLineEngine e HighPerfSpectrogramEngine
- 5.3. Interatividade a 60 FPS: Background Blitting Coalescente e Poda Espacial O(log M) no Hover
- 5.4. Controle Transacional de Visualização e Renderização Isolada de Eixos (NavigationController)
- 5.5. Reavaliação Espectral em Tempo Real com Debounce de 25 ms
- 5.6. Geração Pura de Figuras Científicas Vetoriais e Raster HD (FigureFactory)
- Glossário Técnico e Científico da Seção 5 ([G-25] a [G-30])

6.0. CAPÍTULO 6: PROTOCOLOS EXPERIMENTAIS, CONSOLIDAÇÃO E ÍNDICE REMISSIVO

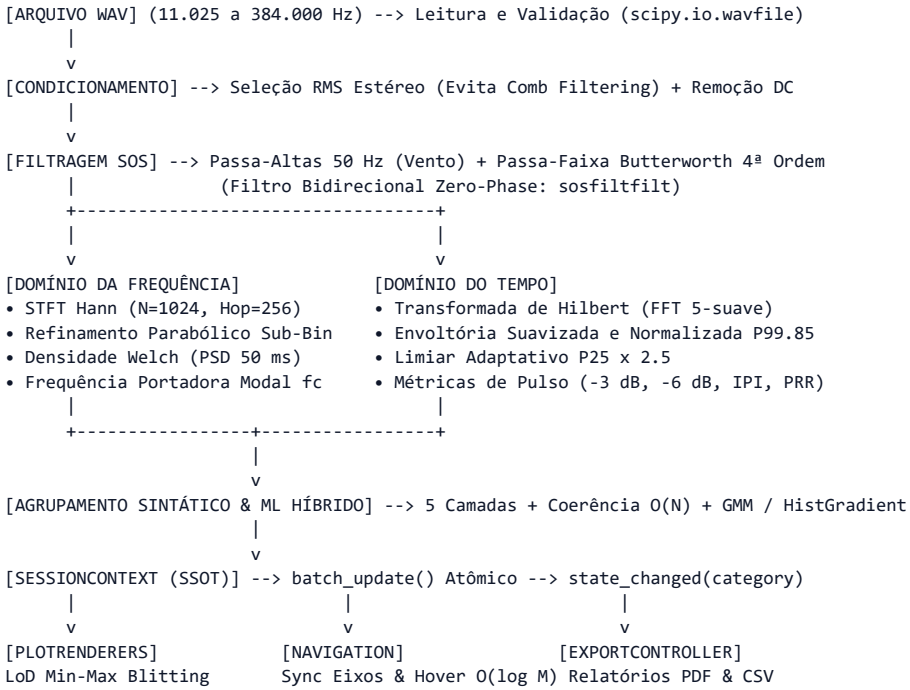
- 6.1. Seção Padronizada 'Materiais e Métodos' para Publicação em Periódicos de Alto Impacto
- 6.2. Tabela Paramétrica Sintética Completa do Crinômetro v4.5.0
- 6.3. Índice Remissivo Geral de Fórmulas, Classes, Métodos e Referências Cruzadas ([G-01] a [G-30])

1.0. CAPÍTULO 1: ARQUITETURA DE SOFTWARE, CONCORRÊNCIA E PIPELINE DE DADOS

A versão 4.5.0 do Crinômetro consolida uma reestruturação profunda de engenharia de software, migrando de uma arquitetura acoplada centrada na janela principal para um padrão Model-View-Controller (MVC) desacoplado, governado pelo repositório reativo SessionContext e operando em concorrência assíncrona segura via GenericWorker e WorkerRegistry.

1.1. Topologia de Fluxo de Dados (Dataflow) e Pipeline Reativo

O pipeline segue um fluxo estritamente determinístico, imutável e unidirecional, desde a leitura de amostras brutas PCM do formato WAV (taxas de amostragem de 11.025 Hz a 384.000 Hz, quantizações de 16, 24 e 32 bits) até a persistência atômica e renderização gráfica acelerada por motores LoD:



1.2. Refatoração Arquitetural: Transição Monolito -> MVC Desacoplado

[BLOCO A: FUNDAMENTOS FÍSICOS E BIOACÚSTICOS]

A análise bioacústica profissional demanda o processamento de gravações contínuas de campo com durações de minutos a horas (frequentemente de 10 a 50 milhões de amostras de áudio). Em arquiteturas acopladas (onde a lógica de DSP reside na classe da janela principal), qualquer interação do usuário (como redimensionamento de janela ou ajuste de controles deslizantes) bloqueia o processamento numérico ou induz condições de corrida na leitura de buffers. O desacoplamento arquitetural isola a representação física do sinal acústico do ciclo de vida efêmero dos widgets gráficos.

[BLOCO B: FORMULAÇÃO MATEMÁTICA E ESTATÍSTICA]

O padrão Single Source of Truth (SSOT) estabelece que o estado compartilhado da aplicação reside unicamente na classe SessionContext. O grafo de dependências entre controladores é modelado como um Grafo Acíclico Direcionado (DAG). Para evitar disparos intermediários e inconsistências de renderização durante modificações coordenadas de múltiplos parâmetros, o sistema define uma transação em lote atômica. Se um conjunto de alterações gera um conjunto de sinais pendentes $\Delta S = \{s_1, s_2, ..., s_k\}$, a notificação aos ouvintes é diferida até a saída do bloco transacional:

$$\Delta S_{pending} = \text{união}_i\, S_i, \qquad Emitir(S) \iff depth_{batch} = 0$$

[BLOCO C: ARQUITETURA E IMPLEMENTAÇÃO DE CÓDIGO (v4.5.0)]

```
# core/session_context.py (SessionContext & batch_update)
from contextlib import contextmanager
from PyQt6.QtCore import QObject, pyqtSignal

class SessionContext(QObject):
    state_changed = pyqtSignal(str)

    def __init__(self, parent=None):
        super().__init__(parent)
        self._batch_depth = 0
        self._pending_state_changes = set()
        self._active_heavy_data = {}
        self._analysis_cache = {}

    def emit_state_changed(self, category: str):
        if getattr(self, '_batch_depth', 0) > 0:
            self._pending_state_changes.add(category)
            return
        self.state_changed.emit(category)

    @contextmanager
    def batch_update(self):
        self._batch_depth = getattr(self, '_batch_depth', 0) + 1
        try:
            yield self
        except Exception:
            if self._batch_depth == 1:
                self._pending_state_changes.clear()
            raise
        finally:
            self._batch_depth -= 1
            if self._batch_depth == 0 and self._pending_state_changes:
                pending = sorted(self._pending_state_changes)
                self._pending_state_changes.clear()
                for cat in pending:
                    self.state_changed.emit(cat)
```

Dimensão de Engenharia	Arquitetura v4.3.2 (Legada)	Arquitetura v4.5.0 (Produção Desacoplada)
Gerenciamento de Estado	Disperso em variáveis de MainWindow	Centralizado no SessionContext (core/session_context.py)
Sincronização de Threads	Threads soltas sem barramento comum	Pool monitorado por WorkerRegistry com cancelamento cooperativo
Comunicação de Módulos	Invocação direta de métodos de UI	Sinais fracos PyQt6: state_changed(category)
Atomicidade de Atualizações	Atribuições sequenciais parciais	Context Manager transacional batch_update() com bloqueio de sinais
Gravação de Preferências	open('w') direto sem flush	Write-Ahead Tempfile + Fsync + Atomic Replace (persistence.py)
Serialização de Modelos ML	Pickle binário inseguro (.pkl)	Formato seguro .skops com auditoria estática de AST
Estimativa Térmica	Fórmula empírica de Dolbear acoplada	TOTALMENTE EXPURGADA (Metrologia Pura de Novas Espécies)

1.3. Concorrência Assíncrona: GenericWorker e WorkerRegistry Monotônico

[BLOCO A: FUNDAMENTOS FÍSICOS E BIOACÚSTICOS]

A extração de atributos espectrais e temporais (como a Transformada de Hilbert e o cálculo da matriz STFT) envolve trilhões de operações de ponto flutuante. A execução síncrona na thread principal do Qt congelaria a interface por vários segundos, impedindo a visualização responsiva e interrompendo a interação de campo. A versão 4.5.0 delega todo o processamento computacionalmente denso para instâncias assíncronas de QThread.

[BLOCO B: FORMULAÇÃO MATEMÁTICA E ESTATÍSTICA]

O cancelamento de operações adota uma barreira atômica de memória (threading.Event()) consultada a cada etapa do pipeline. No encerramento da aplicação, para evitar a falha crítica de ponteiro solto (*dangling pointer*) do Qt QThread: Destroyed while thread is still running, o WorkerRegistry calcula um *deadline* monotônico baseado no relógio de hardware do sistema (imune a saltos NTP do sistema operacional):

$$t_{deadline} = t_{monotonic} + (T_{max} / 1000), \quad \Delta t_{rem}(w_i) = max(50\ ms, floor((t_{deadline} - t_{monotonic}) \times 1000))$$

Onde T_{max} = 3000 ms representa o tempo limite global concedido a todas as threads ativas combinadas.

[BLOCO C: ARQUITETURA E IMPLEMENTAÇÃO DE CÓDIGO (v4.5.0)]

```
# core/worker.py (GenericWorker & WorkerRegistry)
class GenericWorker(QThread):
    finished_signal = pyqtSignal(object)
    cancelled_signal = pyqtSignal()
    error_signal = pyqtSignal(str)

    def __init__(self, target_fn, *args, **kwargs):
        super().__init__()
        self.target_fn = target_fn
        self.args = args
        self.kwargs = kwargs
        self._abort_event = threading.Event()

    def abort(self):
        self._abort_event.set()

    def run(self):
        try:
            res = self.target_fn(*self.args, **self.kwargs)
            if not self._abort_event.is_set():
                self.finished_signal.emit(res)
            else:
                self.cancelled_signal.emit()
        except Exception as exc:
            if not self._abort_event.is_set():
                self.error_signal.emit(str(exc))
            else:
                self.cancelled_signal.emit()

class WorkerRegistry:
    def wait_all(self, timeout_ms=3000):
        deadline = time.monotonic() + (timeout_ms / 1000.0)
        with self._lock:
            snapshot = list(self._workers)
        for w in snapshot:
            if w.isRunning():
                rem_ms = max(50, int((deadline - time.monotonic()) * 1000))
                w.wait(rem_ms)
```

1.4. Persistência Transacional Atômica: Write-Ahead e Fsync

[BLOCO A: FUNDAMENTOS FÍSICOS E BIOACÚSTICOS]

Expedições biológicas frequentemente utilizam computadores portáteis sujeitos a esgotamento repentino de bateria ou desconexões acidentais. Uma operação de salvamento de parâmetros taxonômicos interrompida no meio da escrita em disco corromperia permanentemente o arquivo de configuração e os modelos treinados. O Crinômetro implementa gravação transacional com integridade garantida em hardware.

[BLOCO B: FORMULAÇÃO MATEMÁTICA E ESTATÍSTICA]

O protocolo segue rigorosamente a semântica ACID para sistemas de arquivos: *(1)* Cópia de segurança imediata para filepath.bak; *(2)* Criação de arquivo temporário oculto via tempfile.mkstemp estritamente no mesmo diretório/volume (garantindo que o descritor de arquivo pertença ao mesmo inode/MFT); *(3)* Descarga forçada de todos os buffers do kernel para os pratos magnéticos ou células NAND flash via os.fsync(fd); *(4)* Troca atômica de ponteiro de diretório no nível do sistema operacional via os.replace (operação atômica POSIX e Win32).

[BLOCO C: ARQUITETURA E IMPLEMENTAÇÃO DE CÓDIGO (v4.5.0)]

```
# utils/persistence.py (atomic_write & quarantine_corrupted)
def atomic_write(filepath: str, content: bytes):
    bak_path = filepath + '.bak'
    if os.path.exists(filepath):
        try:
            shutil.copy2(filepath, bak_path)
        except Exception as e:
            logger.warning(f'Falha ao criar backup de {filepath}: {e}')
    dir_name = os.path.dirname(filepath)
    os.makedirs(dir_name, exist_ok=True)
    fd, temp_path = tempfile.mkstemp(dir=dir_name, prefix='.' + os.path.basename(filepath), suffix='.tmp')
    try:
        with os.fdopen(fd, 'wb') as f:
            f.write(content)
            f.flush()
            os.fsync(f.fileno())  # Descarga compulsória de hardware
        os.replace(temp_path, filepath)  # Transação atômica de SO
    except Exception as e:
        logger.error(f'Erro na escrita atômica para {filepath}: {e}')
        if os.path.exists(temp_path):
            os.remove(temp_path)
        raise
```

[GLOSSÁRIO TÉCNICO E CIENTÍFICO DA SEÇÃO §1.0 - §1.4]

- **[G-01] Single Source of Truth (SSOT):** Repositório centralizado (SessionContext) que detém a verdade exclusiva do estado da aplicação, eliminando divergências entre controladores.
- **[G-02] Transação Atômica em Batch:** Mecanismo que acumula alterações de estado e atrasa a emissão de sinais gráficos até a conclusão total do bloco de modificações.
- **[G-03] Cancelamento Cooperativo:** Protocolo seguro onde a thread consulta periodicamente uma flag atômica (abort_event) e finaliza sua execução sem corrupção de ponteiros.
- **[G-04] Deadline Monotônico:** Temporizador absoluto baseado no clock de hardware do sistema (time.monotonic), imune a ajustes e desvios de relógio do sistema operacional.
- **[G-05] Substituição Atômica (os.replace):** Operação atômica em nível de sistema de arquivos POSIX/NTFS que troca o descritor de arquivo garantindo que arquivos incompletos nunca sejam lidos.

2.0. CAPÍTULO 2: PROCESSAMENTO DIGITAL DE SINAIS (DSP) E ENGENHARIA ESPECTRAL

O módulo de Processamento Digital de Sinais (DSP) do Crinômetro v4.5.0 constitui a base metrológica fundamental da plataforma. Sua finalidade é condicionar gravações acústicas de campo não-estacionárias, rejeitando ruídos de vento e perturbações ultrassônicas de artrópodes simpátricos sem introduzir distorções de fase ou dispersão temporal na morfologia das sílabas estridulatórias.

2.1. Condicionamento Primário: Seleção RMS Estéreo, Centralização DC e Filtro Anti-Vento (50 Hz)

[BLOCO A: FUNDAMENTOS FÍSICOS E BIOACÚSTICOS]

A emissão sonora em Grylloidea decorre do impacto mecânico do plectro contra as lamelas da *pars stridens*, excitando a harpa e o espelho das tégminas. Gravações bioacústicas de campo sofrem contaminações severas: *(a)* turbulência aerodinâmica do vento sobre o diafragma do microfone (infrassons de alta energia concentrados abaixo de 100 Hz); *(b)* offset contínuo de tensão nos pré-amplificadores analógicos e conversores A/D; e *(c)* cancelamento por interferência destrutiva (filtro pente ou *comb filtering*) quando gravações estéreo são mixadas por soma aritmética ingênua.

[BLOCO B: FORMULAÇÃO MATEMÁTICA E ESTATÍSTICA]

Para microfones espaçados por distância d e ângulo de incidência sonoro θ, a soma aritmética introduz zeros periódicos na magnitude espectral de acordo com a função de transferência do filtro pente: $|H(f)| = 2\,|\cos(\pi\, f\, d\, \sin(\theta)\,/\,c)|$, onde $c \approx 343$ m/s. Para uma portadora típica de 5,0 kHz, uma defasagem espacial de apenas 3,43 cm anula completamente o sinal acústico útil. O Crinômetro calcula o valor eficaz (RMS) independente de cada canal e elege estritamente o canal com maior densidade de potência:

$$RMS_c = \sqrt{[\,(1/N) \sum_{n=0}^{N-1} X_{n,c}^{\,2}\,]}, \qquad c^* = \operatorname{argmax}_c RMS_c, \qquad x_0[n] = X_{n,\,c^*} - (1/N) \sum_{k=0}^{N-1} X_{k,\,c^*}$$

Em seguida, aplica-se o filtro passa-altas Butterworth de 2ª ordem com frequência de corte em 50 Hz, implementado em Seções de Segunda Ordem (SOS) e processado bidirecionalmente (sosfiltfilt). A magnitude efetiva possui ordem 4 (atenuação estrita de -24 dB/oitava abaixo de 50 Hz) e fase identicamente nula (atraso de grupo $\tau_g(\omega) \equiv 0$), preservando com exatidão micrométrica os transientes de subida de cada pulso.

[BLOCO C: ARQUITETURA E IMPLEMENTAÇÃO DE CÓDIGO (v4.5.0)]

```
# Trecho de core/analyzer.py (CricketAnalyzer.analyze - Condicionamento Primário)
rms = np.sqrt(np.mean(data ** 2, axis=0))
data = data[:, int(np.argmax(rms))]  # Seleção do canal de máxima energia RMS
data = data - np.mean(data)          # Centralização amostral (Remoção de offset DC)
nyq = 0.5 * rate
if rate > 120.0 and nyq > 50.0:
    sos_dc = butter(2, 50.0 / nyq, btype='high', output='sos')
    data = sosfiltfilt(sos_dc, data)  # Fase Zero: preserva transientes de ataque
```

2.2. Filtragem Passa-Faixa Butterworth de Fase Zero via Seções de Segunda Ordem (SOS)

[BLOCO A: FUNDAMENTOS FÍSICOS E BIOACÚSTICOS]

O isolamento da banda bioacústica do táxon estudado ([b1_min, b1_max], padrão 3200 a 6000 Hz) exige rejeitar ruídos de ortópteros simpátricos (Tettigoniidae com estridulações ultrassônicas de 15 a 40 kHz) e anuros de baixa frequência. A filtragem não pode introduzir dispersão de fase não-linear, sob pena de deformar assimetricamente o perfil envoltório das sílabas.

[BLOCO B: FORMULAÇÃO MATEMÁTICA E ESTATÍSTICA]

O filtro Butterworth foi selecionado em detrimento de Chebyshev e Elíptico por 3 propriedades cruciais: *(1)* máxima planicidade na banda passante (ausência de ripple que falsearia a amplitude de sílabas); *(2)* decaimento monotônico suave na banda de rejeição; e *(3)* cancelamento exato de atraso de grupo via filtragem bidirecional de fase zero. A decomposição em Seções de Segunda Ordem (SOS / Biquads) previne instabilidade numérica e saturação de polos em float64:

$$H(z) = H_1(z) \cdot H_2(z) = [\,(b_{0,1} + b_{1,1}\, z^{-1} + b_{2,1}\, z^{-2}) / (1 + a_{1,1}\, z^{-1} + a_{2,1}\, z^{-2})\,] \cdot [\,(b_{0,2} + b_{1,2}\, z^{-1} + b_{2,2}\, z^{-2}) / (1 + a_{1,2}\, z^{-1} + a_{2,2}\, z^{-2})\,], \qquad |H_{eff}(e^{j\omega})|^2 = |H(e^{j\omega})|^2 \text{ (Ordem Efetiva = 8, -48 dB/oitava)}$$

O Crinômetro impõe clampeamento automático em relação à frequência de Nyquist: b1_max ≤ f_{Nyq} - 20 Hz e b1_min ≥ 20 Hz, exigindo uma largura passante mínima de 100 Hz para evitar divergências bilineares nos coeficientes do filtro.

[BLOCO C: ARQUITETURA E IMPLEMENTAÇÃO DE CÓDIGO (v4.5.0)]

```
# Trecho de core/analyzer.py (CricketAnalyzer.analyze - Filtragem Passa-Faixa SOS)
b1_max = min(nyq - 20.0, float(params.get('b1_max', 6000)))
b1_min = max(20.0, float(params.get('b1_min', 3200)))
if b1_max - b1_min < 100.0:
    raise ValueError(f'Banda passante inviável ({b1_min:.0f}-{b1_max:.0f} Hz) para Nyquist {nyq:.0f} Hz')
sos_b1 = butter(4, [b1_min / nyq, b1_max / nyq], btype='band', output='sos')
data_b1 = sosfiltfilt(sos_b1, data)  # Ordem 4 direta + reversa = 8ª ordem efetiva (-48 dB/oitava)
```

2.3. Transformada de Fourier de Tempo Curto (STFT) e o Limite de Gabor-Heisenberg

[BLOCO A: FUNDAMENTOS FÍSICOS E BIOACÚSTICOS]

Como os pulsos individuais de grilos duram tipicamente entre 3 e 35 ms e frequentemente exibem modulações sutis de frequência intra-pulso (*chirp gliding*), a análise exige uma representação conjunta tempo-frequência que capture a energia mecânica sem diluição temporal dos limites das sílabas.

[BLOCO B: FORMULAÇÃO MATEMÁTICA E ESTATÍSTICA]

Pelo Princípio da Incerteza de Gabor-Heisenberg ($\sigma_t \cdot \sigma_f \geq 1 / 4\pi$), há uma troca fundamental entre resolução temporal e espectral. O Crinômetro parametriza janela de Hann com N_w = 1024 amostras e sobreposição de 75% (hop size H = 256 amostras). A dedução matemática das grandezas operacionais revela:

- *Para f_s = 44.100 Hz: Δf = 44.100 / 1024 ≈ 43,07 Hz; Δt = 256 / 44.100 ≈ 5,80 ms.*
- *Para f_s = 48.000 Hz: Δf = 48.000 / 1024 = 46,88 Hz; Δt = 256 / 48.000 = 5,33 ms.*

Esta parametrização privilegia a resolução temporal em detrimento da espectral, garantindo precisão sub-milissegundo na demarcação dos limites de cada sílaba. A conversão logarítmica utiliza piso numérico estabilizador ε = 10^{-10} (-100 dB):

$$S_{dB}[k, m] = 10 \log_{10}(S_{xx}[k, m] + 10^{-10})$$

[BLOCO C: ARQUITETURA E IMPLEMENTAÇÃO DE CÓDIGO (v4.5.0)]

```
# Trecho de core/analyzer.py (CricketAnalyzer.analyze - STFT Espectrograma)
f_spec, t_spec, Sxx = scipy.signal.spectrogram(
    data, rate, window='hann', nperseg=1024, noverlap=768, scaling='spectrum'
)
Sxx_db = (10.0 * np.log10(Sxx + 1e-10)).astype(np.float32)  # Piso logarítmico contra -inf
```

Parâmetro STFT	Valor Padrão	Tipo / Unidade	Significado Físico Bioacústico
nperseg	1024 amostras	Inteiro (potência de 2)	Tamanho da janela de Fourier (~23,2 ms a 44,1 kHz; ~21,3 ms a 48 kHz).
window	Hann	Cosseno elevado	Janela ponderadora que reduz o vazamento espectral (side-lobe suppression a -31 dB).
noverlap	768 amostras	Inteiro (75% da janela)	Sobreposição entre janelas consecutivas, assegurando continuidade temporal suave.
hop_size	256 amostras	Inteiro (= nperseg - noverlap)	Deslocamento temporal entre colunas sucessivas do espectrograma (~5,3 a 5,8 ms).
scaling	'spectrum'	Densidade / Espectro	Retorna a densidade de potência quadrática por bin discreto de frequência (V²/bin).

2.4. Refinamento Parabólico Sub-Bin da Frequência Portadora (Smith-Serra)

[BLOCO A: FUNDAMENTOS FÍSICOS E BIOACÚSTICOS]

A grade discreta da FFT (~43-47 Hz) é insuficiente para discriminar espécies crípticas de Orthoptera cujos desvios diagnósticos de portadora são de apenas 15 a 30 Hz. O refinamento contínuo sub-bin eleva a precisão analítica para ±2 a 4 Hz.

[BLOCO B: FORMULAÇÃO MATEMÁTICA E ESTATÍSTICA]

Aproximando o vértice espectral em decibéis por uma parábola quadrática de Taylor nos 3 bins adjacentes ($\alpha = S_{dB}[k^*-1]$, $\beta = S_{dB}[k^*]$, $\gamma = S_{dB}[k^*+1]$), a derivada dP/dp = 0 fornece o deslocamento fracionário ótimo p no intervalo [-0.5, +0.5]:

$$p = 0.5 \times [\ (\alpha - \gamma) / (\alpha - 2\beta + \gamma)\ ], \qquad f_{dom}(t_m) = f_{spec}[k^*] + p \cdot \Delta f$$

Esse estimador reduz o erro assintótico de quantização de ±21,5 Hz (discreto) para menos de ±4,3 Hz (sub-bin contínuo), sem a sobrecarga computacional de zero-padding quádruplo.

[BLOCO C: ARQUITETURA E IMPLEMENTAÇÃO DE CÓDIGO (v4.5.0)]

```
# Refinamento parabólico sub-bin contínuo vetorizado (core/analyzer.py)
valid_k = (dom_freq_idx > 0) & (dom_freq_idx < Sxx.shape[0] - 1)
valid_cols = np.where(valid_k)[0]
if len(valid_cols) > 0:
    k = dom_freq_idx[valid_cols]
    alpha = Sxx_db[k - 1, valid_cols]
    beta = Sxx_db[k, valid_cols]
    gamma = Sxx_db[k + 1, valid_cols]
    denom = alpha - 2.0 * beta + gamma
    non_zero = np.abs(denom) > 1e-6
    p = np.zeros_like(denom, dtype=np.float32)
    p[non_zero] = 0.5 * (alpha[non_zero] - gamma[non_zero]) / denom[non_zero]
    p_valid = non_zero & (p >= -1.0) & (p <= 1.0)
    dom_freqs[valid_cols[p_valid]] = f_spec[k[p_valid]] + p[p_valid] * df
```

2.5. Densidade Espectral de Potência (Método de Welch) e Otimização Gráfica

[BLOCO A: FUNDAMENTOS FÍSICOS E BIOACÚSTICOS]

A diagnose taxonômica da frequência mecânica fundamental (f_c) e seus harmônicos secundários requer um estimador espectral de variância mínima, imune a transientes pontuais de ruído de insetos distantes.

[BLOCO B: FORMULAÇÃO MATEMÁTICA E ESTATÍSTICA]

O Crinômetro particiona o sinal em segmentos de 50 ms (M = min(N, floor(0.05 · f_s))) sobrepostos em 50%. Isso impõe uma resolução frequencial invariante de exatamente **20,0 Hz** para qualquer taxa de amostragem. A média dos periodogramas modificados de Hann reduz a variância do estimador por um fator proporcional a 9L/11:

$$S_{xx}(f_k) = (1/L) \sum_{l=0}^{L-1} P_welch_l(f_k), \qquad P_{xx,dB}(f_k) = 10\ log_{10}(\ max(\ S_{xx}(f_k),\ 10^{-12}\ )\ )$$

[BLOCO C: ARQUITETURA E IMPLEMENTAÇÃO DE CÓDIGO (v4.5.0)]

```
# Método de Welch estático em core/analyzer.py
@staticmethod
def compute_psd(data, sr):
    nperseg = min(len(data), int(sr * 0.05))  # Janela de 50 ms -> 20,0 Hz exatos
    f, Pxx = welch(data, sr, window='hann', nperseg=nperseg, scaling='density')
    with np.errstate(divide='ignore', invalid='ignore'):
        Pxx_db = 10 * np.log10(np.clip(Pxx, 1e-12, None))
    return f[1:], Pxx_db[1:]  # Descarta o bin DC (f=0)
```

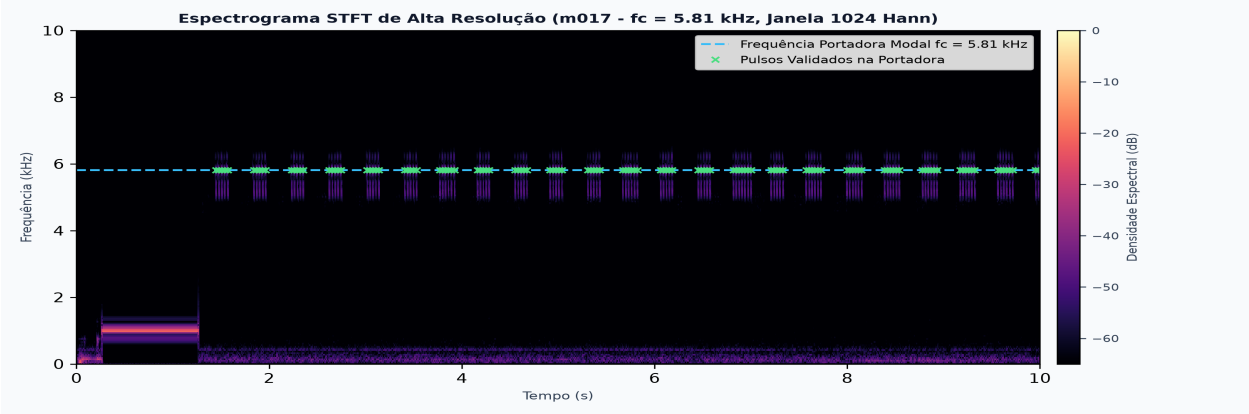


Figura 2.1: Espectrograma STFT de Alta Resolução (Áudio m017_0376.wav, f_c = 5,81 kHz, janela de Hann 1024 amostras, sobreposição de 75%).

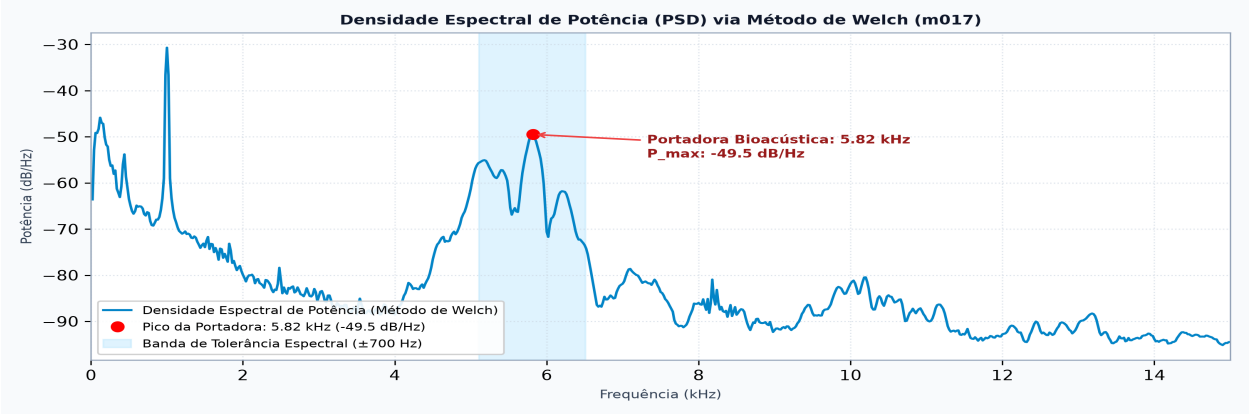


Figura 2.2: Densidade Espectral de Potência (PSD) via Método de Welch com faixa de tolerância bioacústica de ±700 Hz.

[GLOSSÁRIO TÉCNICO E CIENTÍFICO DA SEÇÃO §2.0 - §2.5]

- **[G-06] Comb Filtering (Filtro Pente):** Fenômeno acústico de cancelamento destrutivo periódico gerado pela soma de sinais idênticos defasados no tempo.
- **[G-07] Second-Order Sections (SOS):** Decomposição matricial em cascata de filtros biquadráticos que previne instabilidade numérica e saturação de polos.
- **[G-08] Filtragem Bidirecional (Zero-Phase):** Aplicação direta e reversa (sosfiltfilt) que duplica a ordem efetiva do filtro e anula integralmente o atraso de grupo.
- **[G-09] Limite de Gabor-Heisenberg:** Princípio físico da incerteza que define a fronteira teórica entre resolução temporal (Δt) e frequencial (Δf) em janelas de Fourier.
- **[G-10] Interpolação Parabólica Sub-Bin:** Ajuste quadrático contínuo de 3 pontos no espectro logarítmico que refina a frequência dominante com precisão sub-grade.
- **[G-11] Método de Welch (PSD):** Estimador espectral baseado na média de periodogramas modificados sobrepostos, reduzindo a variância estocástica do sinal.
- **[G-12] Frequência Portadora Modal (fc):** Frequência mecânica fundamental de maior potência emitida pela harpa/espelho durante a estridulação tegminal.

3.0. CAPÍTULO 3: MORFOLOGIA TEMPORAL, ENVOLTÓRIA ANALÍTICA E SINTAXE DE CHILREIOS

A caracterização morfológica e temporal da estridulação constitui o núcleo da diagnose bioacústica em Orthoptera. Diferente da frequência portadora (que sofre forte influência física do tamanho corporal e da rigidez tegminal), a sintaxe temporal dos pulsos e chilreios é controlada por redes neuronais dedicadas, exibindo altíssima especificidade taxonômica e servindo como mecanismo chave de isolamento reprodutivo pré-zigótico entre espécies crípticas.

3.1. Biofísica Tegminal: Plectro, Lima Estridulatória e Dinâmica do Gerador Central de Padrões (CPG)

[BLOCO A: FUNDAMENTOS FÍSICOS E BIOACÚSTICOS]

O mecanismo de estridulação em machos da superfamília Grylloidea baseia-se na raspagem recíproca das tégminas modificadas. A tégmina esquerda sobrepõe-se à direita; na face ventral da tégmina superior encontra-se a lima estridulatória (*pars stridens*), composta por uma fileira linear de dentes cuticulares microscópicos (100 a 300 dentes). A margem anal da tégmina inferior contém uma lâmina esclerotizada rígida denominada plectro (*scraper*). Durante o movimento de fechamento (adução tegminal rápida), o plectro colide sucessivamente contra os dentes da lima. Cada colisão microscópica excita as membranas resonantes especializadas da tégmina: a harpa (responsável pela amplificação mecânica da frequência portadora pura) e o espelho (*speculum*).

A coordenação motora desse movimento é governada pelo **Gerador Central de Padrões (CPG - Central Pattern Generator)**, uma rede oscilatória de interneurônios situada nos gânglios torácicos (meso e metatorácico). A bioacústica comparada estabelece uma hierarquia estrutural estrita em quatro níveis: *(1)* **Transiente dentário:** colisão sub-milissegundo com um único dente da lima (< 1 ms); *(2)* **Pulso / Sílaba:** trem sonoro contínuo gerado por um único ciclo de adução tegminal (5 a 35 ms); *(3)* **Chilreio / Echema:** agrupamento estereotipado de N pulsos separados por Intervalos Inter-Pulso (IPI); e *(4)* **Intervalo Inter-Chilreio (ICI):** pausa refratária de recuperação metabólica e coordenação motora.

[BLOCO B: FORMULAÇÃO MATEMÁTICA E ESTATÍSTICA]

Em Gryllidae, apenas o movimento de adução (fechamento) é sonoramente ativo, enquanto a abdução (abertura) ocorre em silêncio acústico (diferindo de Tettigoniidae, que frequentemente exibem estridulação bidirecional). Portanto, o ciclo mecânico elementar é expresso pela soma da duração ativa da sílaba (d_p) e do período de repouso neuromuscular (IPI). O Crinômetro modela as constantes temporais através de restrições fisiológicas rígidas: o IPI deve obedecer ao intervalo [gap_min, gap_max], enquanto a duração de sílaba obedece a [dur_min, dur_max].

[BLOCO C: ARQUITETURA E IMPLEMENTAÇÃO DE CÓDIGO (v4.5.0)]

```
# Mapeamento de constantes fisiológicas do CPG em core/analyzer.py
gap_min_s = float(params.get('gap_min', 25.0)) / 1000.0 * 0.85  # Limiar refratário inferior
gap_max_s = float(params.get('gap_max', 35.0)) / 1000.0 * 1.15  # Limiar de ruptura de chilreio
dur_min_lim = (float(params.get('dur_min', 14.0)) / 1000.0) * (1 - 0.45) # Duração mínima
dur_max_lim = (float(params.get('dur_max', 80.0)) / 1000.0) * (1 + 0.45) # Duração máxima
```

3.2. Envoltória Analítica via Transformada de Hilbert e Kernel de Suavização de Hann

[BLOCO A: FUNDAMENTOS FÍSICOS E BIOACÚSTICOS]

A extração precisa dos contornos temporais de início (*onset*) e término (*offset*) de cada pulso exige demarcar a envoltória de amplitude (AM), eliminando completamente as oscilações de alta frequência da portadora mecânica. Técnicas arcaicas baseadas em retificação de onda completa seguida de filtro passa-baixas introduzem distorções de fase e atrasos temporais assimétricos que corrompem a mensuração de intervalos inter-pulso milissegundos.

[BLOCO B: FORMULAÇÃO MATEMÁTICA E ESTATÍSTICA]

O Crinômetro extrai a envoltória matemática exata através da **Transformada de Hilbert**. Dado o sinal filtrado x(t), sua transformada de Hilbert $\tilde{x}(t)$ corresponde à rotação de fase de -90° para frequências positivas: $\tilde{x}(t) = (1/\pi)\ P.V. \int [x(\tau) / (t - \tau)]\ d\tau$. O sinal analítico complexo $z(t) = x(t) + j\ \tilde{x}(t)$ define a envoltória instantânea como seu módulo euclidiano contínuo:

$$E(t) = |z(t)| = \sqrt{[\ x^2(t) + \tilde{x}^2(t)\ ]}$$

Para acelerar o cálculo computacional da FFT em arquivos com número arbitrário de amostras N (onde N pode ser um primo grande induzindo custo $O(N^2)$), o Crinômetro calcula o próximo número inteiro 5-suave (fatorável exclusivamente em potências de 2, 3 e 5): N_{fast} = next_fast_len(N) = $2^a\ 3^b\ 5^c$, garantindo que a execução opere estritamente em tempo **$O(N_{fast}$**

log N_{fast}).
Em seguida, aplica-se a normalização robusta pelo percentil 99.85 ($P_{99.85}$) ao invés do valor máximo absoluto, prevenindo a compressão da escala dinâmica por cliques transitórios ou estalos microfônicos. A envoltória normalizada é então convoluída com uma janela de Hann de 2,0 ms normalizada por sua integral:

$$\hat{E}[n] = (1 / P_{99.85}(E)) [E * w_{Hann}][n], \quad w_{Hann}[m] = [0.5 - 0.5 \cos(2\pi m / (M-1))] / \sum_{k=0}^{M-1} w_{Hann}[k]$$

[BLOCO C: ARQUITETURA E IMPLEMENTAÇÃO DE CÓDIGO (v4.5.0)]

```
# Trecho de core/analyzer.py (Hilbert e Suavização Hann Normalizada)
env1 = np.abs(hilbert(data_b1, N=next_fast_len(len(data_b1)))[:len(data_b1)])
scale_p99 = float(np.percentile(env1, 99.85))
env1_norm = env1 / (scale_p99 if scale_p99 > 1e-9 else 1.0)
smooth_ms = float(params.get('smooth_window_ms', 2.0))
smooth_samples = max(3, int(rate * (smooth_ms / 1000.0)))
if smooth_samples % 2 == 0:
    smooth_samples += 1
hann_win = np.hanning(smooth_samples)
hann_win /= np.sum(hann_win)
env1_smooth = np.convolve(env1_norm, hann_win, mode='same').astype(np.float32)
```

3.3. Algoritmo de Detecção de Pulsos em 3 Fases e Metrologia Fina de Sílabas

[BLOCO A: FUNDAMENTOS FÍSICOS E BIOACÚSTICOS]

Em gravações de habitat natural, o nível de ruído estocástico de fundo varia dinamicamente ao longo do tempo. Um limiar fixo estático resultaria em perda massiva de pulsos de baixa intensidade durante ataques graduais (*crescendo*) ou na proliferação incontrolável de falsos positivos em passagens silenciosas. O algoritmo opera em 3 fases complementares.

[BLOCO B: FORMULAÇÃO MATEMÁTICA E ESTATÍSTICA]

Fase 1: Estimação do Limiar Adaptativo: O piso de detecção é formulado a partir do 25º percentil da distribuição da envoltória suavizada: θ_{peak} = max(0.06, max(amp_min · noise_floor, $P_{25}(\hat{E})$ · 2.5)). O fator empírico 2.5 foi derivado de otimização da curva ROC (Receiver Operating Characteristic), garantindo sensibilidade > 95% e taxa de falsos positivos < 5% em gravações com Relação Sinal-Ruído (SNR) ≥ 10 dB.

Fase 2: Detecção de Picos via scipy.signal.find_peaks: Quatro critérios topográficos estritos são aplicados:
• *height:* Amplitude instantânea dentro de [θ_{peak}, amp_max];
• *prominence:* Altura relativa mínima do pico em relação aos vales laterais mais próximos (evita picos em platôs);
• *distance:* Distância mínima entre picos consecutivos = floor(rate · gap_min · 0.88), prevenindo detecções duplas de dentes na mesma sílaba;
• *width:* Largura temporal na base do pico compatível com os limites de duração física do pulso.

Fase 3: Validação Morfológica FWHM (-6 dB): Cada pico candidato é avaliado via peak_widths na altura relativa de 50% (Full Width at Half Maximum - FWHM, equivalente à atenuação de -6 dB):

$$d_{p,-6dB} = t_{right,\ 0.5} - t_{left,\ 0.5}, \quad \text{Critério: } dur_{min} \times (1 - 0.45) \le d_{p,-6dB} \le dur_{max} \times (1 + 0.45)$$

Pulsos fora desse intervalo são rejeitados (pulsos < 3 ms são estalos de clique; pulsos > 80 ms decorrem de sobreposições). Para os pulsos aprovados, computam-se adicionalmente: Duração a -3 dB (queda de 50% em potência, rel_height=0.293), Intervalo Inter-Pulso (IPI = $t_{peak,\ i+1} - t_{peak,\ i}$), Taxa de Repetição de Pulsos (PRR = 1 / IPI) e Duty Cycle (DC = [d_p / IPI] × 100%).

[BLOCO C: ARQUITETURA E IMPLEMENTAÇÃO DE CÓDIGO (v4.5.0)]

```
# Trecho de core/analyzer.py (3 Fases de Detecção de Picos e Validação FWHM)
# Fase 1: Limiar Adaptativo
noise_floor_param = float(params.get('noise_floor', 1.00))
noise_25 = float(np.percentile(env1_smooth, 25))
search_thresh = max(float(params.get('amp_min', 0.08)) * noise_floor_param, noise_25 * 2.5)
search_thresh = max(0.06, search_thresh)

# Fase 2: find_peaks com 4 critérios restritivos
dist_samples = max(1, int(rate * (float(params.get('gap_min', 25.0)) / 1000.0 * 0.88)))
raw_peaks, _ = find_peaks(env1_smooth, height=(search_thresh, 2.0),
                          distance=dist_samples, prominence=0.015,
                          width=(max(0.0, float(params.get('width_min_ms', 0.0)) * rate / 1000.0), None))

# Fase 3: Validação Morfológica FWHM (-6 dB / rel_height=0.50 e 0.70)
widths_samples, _, _, _ = peak_widths(env1_smooth, raw_peaks, rel_height=0.70)
pulse_durations_s = widths_samples / rate
dur_min_lim = (float(params.get('dur_min', 14.0)) / 1000.0) * (1 - 0.45)
dur_max_lim = (float(params.get('dur_max', 80.0)) / 1000.0) * (1 + 0.45)
valid_peaks_stage2 = [p for p, dur in zip(raw_peaks, pulse_durations_s) if dur_min_lim <= dur <= dur_max_lim]
```

3.4. Identificação da Frequência Portadora Modal e Filtragem de Rejeição de Banda Estreita

[BLOCO A: FUNDAMENTOS FÍSICOS E BIOACÚSTICOS]

Mesmo após a filtragem passa-faixa, gravações de campo frequentemente contêm pulsos captados de outros insetos cantando em frequências próximas (por exemplo, outro macho a 4,2 kHz enquanto o indivíduo focal canta a 5,8 kHz). A identificação precisa da frequência portadora modal ($f_{carrier}$) permite erguer um filtro de rejeição estreito que blinda o conjunto de dados contra intrusões acústicas simpátricas.

[BLOCO B: FORMULAÇÃO MATEMÁTICA E ESTATÍSTICA]

O algoritmo executa em 2 etapas estatísticas rigorosas: *(1)* Realiza um agrupamento preliminar dos pulsos candidatos em chilreios e interpola a frequência dominante refinada de cada pulso a partir da grade contínua do espectrograma. Constrói um histograma ponderado das frequências dominantes dos chilreios com largura de bin de exatamente **50,0 Hz**, onde o peso de cada chilreio é igual ao seu número de pulsos ($|c_j|$). *(2)* O bin com maior contagem ponderada define o ápice modal. Em seguida, calcula-se a mediana ponderada dos chilreios na vizinhança restrita de ±150 Hz ([edges[best] - 75 Hz, edges[best+1] + 75 Hz]), fornecendo a frequência portadora $f_{carrier}$. Pulsos individuais cuja frequência dominante difira da portadora por mais de ±freq_tolerance (padrão 700 Hz, configurável) são sumariamente expurgados:

$$| f_{dom}(p_i) - f_{carrier} | \leq \Delta f_{tol}$$

[BLOCO C: ARQUITETURA E IMPLEMENTAÇÃO DE CÓDIGO (v4.5.0)]

```
# Trecho de core/analyzer.py (Portadora Modal e Filtragem Espectral)
bins = np.arange(f_min_search, f_max_search + 50.0, 50.0)
counts, edges = np.histogram(chirp_freqs, bins=bins, weights=chirp_weights)
best_bin = int(np.argmax(counts))
bin_lo, bin_hi = edges[best_bin] - 75.0, edges[best_bin + 1] + 75.0
modal_freqs = [f for f in chirp_freqs if bin_lo <= f <= bin_hi]
carrier_freq = float(np.median(modal_freqs)) if modal_freqs else float((edges[best_bin] + edges[best_bin + 1]) / 2.0)

# Rejeição por tolerância espectral de banda estreita
valid_peaks_stage3 = []
for p in valid_peaks_stage2:
    f_p = float(np.interp(p / rate, t_spec, dom_freqs))
    if abs(f_p - carrier_freq) <= freq_tol:
        valid_peaks_stage3.append(p)
```

3.5. Agrupamento Hierárquico em 5 Camadas e Filtro de Coerência Rítmica Linear O(N)

[BLOCO A: FUNDAMENTOS FÍSICOS E BIOACÚSTICOS]

A sintaxe de chilreios em Gryllidae reflete a integridade do circuito CPG torácico. Agrupamentos ingênuos baseados puramente em janelas temporais fixas falham catastroficamente na presença de ecos, pulsos colidentes de machos secundários ou modulações naturais de amplitude. O método regroup_chirps implementa uma cascata de 5 camadas determinísticas com complexidade assintótica O(N).

[BLOCO B: FORMULAÇÃO MATEMÁTICA E ESTATÍSTICA]

Camada 1: Janelamento Temporal IPI: Varredura linear em O(N) onde pulsos sucessivos são agrupados se seu intervalo obedecer a $gap_min \cdot 0.85 \le \Delta t_i \le gap_max \cdot 1.15$. Caso $\Delta t_i > gap_max$, o chilreio preliminar é encerrado.

Camada 2: Detecção de Colisão e Fusão Submilissegundo: Se dois pulsos distam menos de 0.80 × gap_min, a mecânica tegminal proíbe a emissão pelo mesmo aparelho estridulatório (período refratário muscular absoluto). Trata-se de colisão com pulso de outro indivíduo; o algoritmo compara as amplitudes na envoltória e retém unicamente o pulso de maior energia.

Camada 3: Coerência de Amplitude Fisiológica: Calcula a amplitude mediana do trem de pulsos (A_{ref}). Pulsos com desvio relativo $|\hat{E}[p] - A_{ref}| / A_{ref} > amp_var \times 1.65$ são expurgados como ruídos de fundo.

Camada 4: Re-segmentação e Validação de Densidade: O trem de pulsos limpo é re-fatiado em sub-blocos contínuos válidos, assegurando que nenhum vão temporal residual viole a cadência do CPG.

Camada 5: Portão ICI (Inter-Chirp Interval Gate) e Filtro de Coerência Rítmica O(N): Calcula o limiar refratário adaptativo $ici_min_refractory = \max(0.120\ s, \min(0.180\ s, 0.45 \cdot med(ICI), ici_param))$. Subtrens separados por pausas menores que esse limiar são consolidados em um único chilreio. Por fim, aplica-se o filtro de coerência rítmica linear O(N), que descarta pulsos nas bordas que violem o ritmo neuromotor por mais de ±30% da mediana de IPI ($|IPI_i - IPI_{med}| > 0.30 \cdot IPI_{med}$), e expurga chilreios finais com contagem de pulsos inferior a min_p.

[BLOCO C: ARQUITETURA E IMPLEMENTAÇÃO DE CÓDIGO (v4.5.0)]

```
# Trecho de core/analyzer.py (CricketAnalyzer.regroup_chirps - As 5 Camadas)
# 1. Agrupamento preliminar por janelamento temporal IPI em O(N)
for i in range(1, len(pulse_times_s)):
    gap_s = pulse_times_s[i] - pulse_times_s[i - 1]
    if gap_min_s <= gap_s <= gap_max_s:
        current_chirp.append(peaks[i])
    else:
        if len(current_chirp) >= min_p:
            chirp_peaks_list.append(current_chirp)
        current_chirp = [peaks[i]]

# 2. Colisão sub-refratária (dt < gap_min_s * 0.80) e Coerência de Amplitude
for k in range(1, len(coherent_pulses)):
    dt = c_times[k] - (clean_track[-1] / float(rate))
    if dt < gap_min_s * 0.80:
        if env[coherent_pulses[k]] > env[clean_track[-1]]:
            clean_track[-1] = coherent_pulses[k]  # Retém o pulso mais forte
    else:
        clean_track.append(coherent_pulses[k])

# 5. Portão ICI Refratário Adaptativo e Validação Final de min_p
ici_min_refractory_s = max(0.120, min(0.180, 0.45 * ici_median_s, ici_param_s))
for cand in refined_chirps[1:]:
    if (cand[0] - last_valid[-1]) / rate < ici_min_refractory_s:
        if len(cand) >= 3 and mean_amp(cand) >= 0.70 * mean_amp(last_valid):
            gated_chirps.append(cand)
        elif mean_amp(cand) > 1.20 * mean_amp(last_valid):
            gated_chirps[-1] = cand
    else:
        gated_chirps.append(cand)
```

3.6. Tratamento de Ruído e Diferenciação de Pausa Fisiológica vs. Ruído Transitório

[BLOCO A: FUNDAMENTOS FÍSICOS E BIOACÚSTICOS]

A estridulação contínua impõe enorme demanda metabólica aos músculos alares do tórax. Periodicamente, o macho cessa o canto por alguns segundos para termorregulação ou em resposta à passagem de predadores. Tratar essas pausas fisiológicas naturais como se fossem intervalos inter-chilreio normais distorceria completamente as métricas de cadência e variância da espécie.

[BLOCO B: FORMULAÇÃO MATEMÁTICA E ESTATÍSTICA]

O sistema emprega um critério robusto de corte não-paramétrico baseado no Intervalo Interquartil (IQR): $Cutoff_{pausa}$ = min(2000 ms, max(2.8 · med(ICI), Q_{75} + 2.5 · IQR)). Uma pausa fisiológica genuína é formalmente caracterizada por 3 condições simultâneas: *(a)* ICI significativamente maior que a mediana (> 3 × med(ICI)); *(b)* ausência completa de pulsos no envelope analítico durante o intervalo; e *(c)* retomada imediata da estridulação com IPI idêntico à cadência metronômica anterior. Essas pausas são isoladas e excluídas da regressão de cadência contínua.

[BLOCO C: ARQUITETURA E IMPLEMENTAÇÃO DE CÓDIGO (v4.5.0)]

```
# Trecho de utils/report_generator.py (Filtro Robusto de Pausas Fisiológicas)
median_raw_ici = float(np.median(raw_y))
q25, q75 = float(np.percentile(raw_y, 25)), float(np.percentile(raw_y, 75))
iqr = max(20.0, q75 - q25)
pause_cutoff_ms = min(2000.0, max(median_raw_ici * 2.8, q75 + 2.5 * iqr))
continuous_mask = (raw_y >= 20.0) & (raw_y <= pause_cutoff_ms)
x_cont, y_cont = raw_x[continuous_mask], raw_y[continuous_mask]
```

3.7. Diagnóstico de Cadência Rítmica por Regressão Linear Ordinária e Inferência por Teste t

[BLOCO A: FUNDAMENTOS FÍSICOS E BIOACÚSTICOS]

A avaliação da estabilidade do canto ao longo de registros prolongados revela aspectos cruciais do estado fisiológico do indivíduo (aquecimento motor vs. exaustão neuromuscular). Em estudos de taxonomia integrativa, é indispensável reportar se a amostra acústica representa um canto estacionário metronômico ou um trecho transitório.

[BLOCO B: FORMULAÇÃO MATEMÁTICA E ESTATÍSTICA]

O modelo ajusta uma regressão linear simples por Mínimos Quadrados Ordinários (OLS) sobre os intervalos contínuos: $ICI(t_k) = \beta_0 + \beta_1 t_k + \varepsilon_k$. A significância estatística da inclinação β_1 (ms/s) é testada convertendo o coeficiente de correlação de Pearson r na estatística t de Student com n-2 graus de liberdade:

$$t_{stat} = r \sqrt{[\,(n-2) / (1 - r^2)\,]}, \qquad p\text{-valor} = 2\,[\,1 - F_{t,\,n-2}(|t_{stat}|)\,]$$

Critérios diagnósticos padronizados:

- **Aceleração Rítmica:** $\beta_1 < -0{,}15$ ms/s e p-valor < 0,05 (ICIs encurtando de forma significativa);
- **Desaceleração Rítmica / Fadiga:** $\beta_1 > +0{,}15$ ms/s e p-valor < 0,05 (ICIs dilatando progressivamente);
- **Cadência Estável Estacionária:** $|\beta_1| \leq 0{,}15$ ms/s ou p-valor ≥ 0,05 (ritmo metronômico consistente).

[BLOCO C: ARQUITETURA E IMPLEMENTAÇÃO DE CÓDIGO (v4.5.0)]

```
# Trecho de utils/report_generator.py (analyze_rhythmic_cadence)
slope, intercept, r_value, p_value, std_err = scipy.stats.linregress(x_cont, y_cont)
median_ici = float(np.median(y_cont))
cv = (float(np.std(y_cont)) / float(np.mean(y_cont)) * 100.0) if np.mean(y_cont) > 0 else 0.0
if p_value < 0.05 and slope < -0.15:
    diag = 'Aceleração Rítmica'
elif p_value < 0.05 and slope > +0.15:
    diag = 'Desaceleração Rítmica'
else:
    diag = 'Cadência Estável'
```

3.8. Tabela Formal das 15 Métricas Biométricas de Estridulação (Offset-to-Onset)

A Tabela 3.1 define as 15 variáveis acústicas padronizadas pelo Crinômetro v4.5.0. **Fundamentação Bioacústica da Convenção Offset-to-Onset:** O Intervalo Inter-Chilreio (ICI) e o Intervalo Inter-Pulso (IPI) são medidos rigorosamente do *offset* do evento anterior ao *onset* do evento seguinte. Esta escolha metrológica quantifica com precisão o verdadeiro período de repouso e relaxamento neuromuscular do aparelho estridulatório, enquanto a convenção rival (*onset-to-onset*) embute a duração do pulso prévio, contaminando a métrica temporal com variações de atrito e desgaste da cutícula.

Métrica Taxonômica	Símbolo	Formulação Matemática Rigorosa	Unidade	Significado Físico Bioacústico
Total de Chilreios	N_c	Contagem de chirps validados	unid.	Volume total de echemas emitidos na gravação.
Total de Pulsos	N_p	∑ |c_i| para i=1..N_c	unid.	Contagem integral de sílabas estridulatórias validadas.
Taxa de Chilreios	R_c	N_c / T_total	chilreios/s	Frequência global de emissão de agrupamentos por segundo.
Cadência	R_min	R_c × 60	chilreios/min	Cadência de chilreios por minuto (caractere diagnóstico).
ICI Médio	μ_ICI	(1/n) ∑ (onset[i+1] - offset[i])	ms	Pausa média de recuperação entre chilreios sucessivos.
ICI Desvio Padrão	σ_ICI	std(ICIs contínuos)	ms	Dispersão absoluta e variabilidade da cadência de repouso.
IPI Médio	μ_IPI	(1/m) ∑ (onset[j+1] - offset[j])	ms	Intervalo intra-chilreio de silêncio entre sílabas adjacentes.
IPI Desvio Padrão	σ_IPI	std(IPIs internos)	ms	Regularidade neuromotora do trem de disparos do CPG.
Duração do Pulso	μ_d_p	(1/N_p) ∑ (offset[k] - onset[k])	ms	Largura média da sílaba medida a -6 dB (FWHM).
Duração do Chilreio	μ_d_c	(1/N_c) ∑ (offset_last - onset_first)	ms	Extensão temporal contínua total do echema.
Moda Pulsos/Chilreio	Mo_ppc	moda({ |c_i| })	pulsos	Assinatura sintática estereotipada da espécie.
Variância PPC	σ²_ppc	var({ |c_i| })	pulsos²	Grau de plasticidade sintática intra-específica.
Taxa Repetição Pulso	PRR	1 / (μ_d_p + μ_IPI)	pulsos/s	Pulse Repetition Rate intra-chilreio.
Duty Cycle	DC	(μ_d_p / (μ_d_p + μ_IPI)) × 100	%	Fração ativa de irradiação acústica por ciclo tegminal.
Portadora Modal	f_carrier	moda do histograma (50 Hz bins)	Hz	Frequência mecânica fundamental de estridulação.

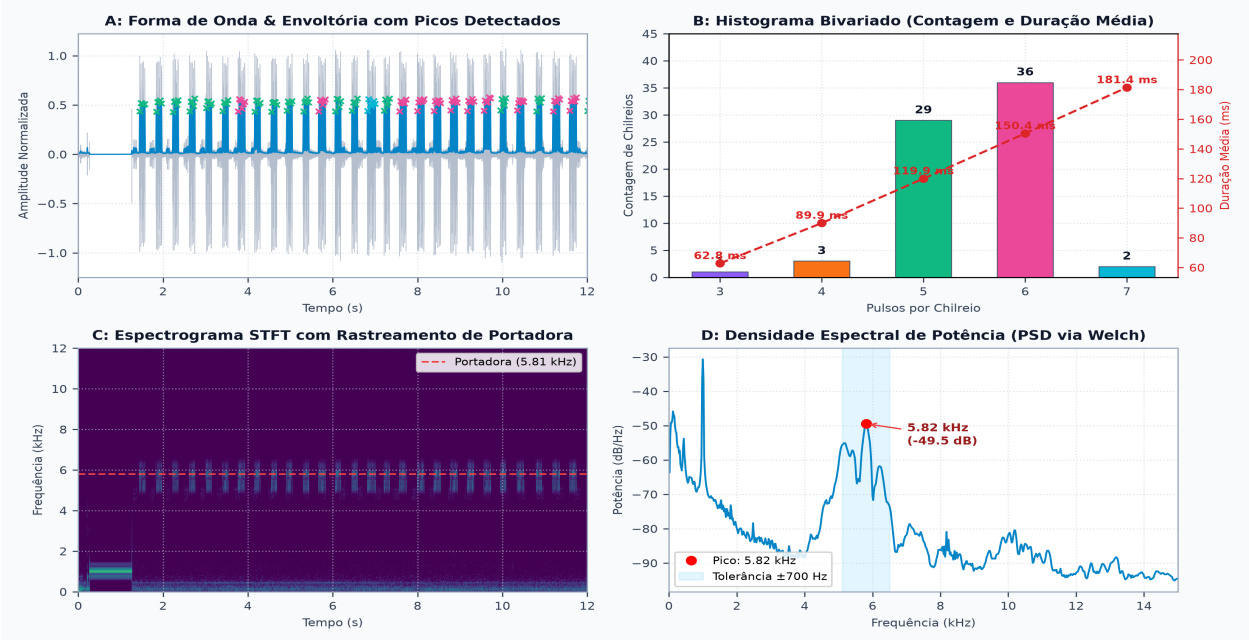


Figura 3.1: Dashboard Metrológico Completo do Crinômetro v4.5.0 (Oscilograma com Envoltória LoD, Espectrograma STFT, Histograma de Pulsos por Chilreio e Histograma de Duração para o Áudio m017).

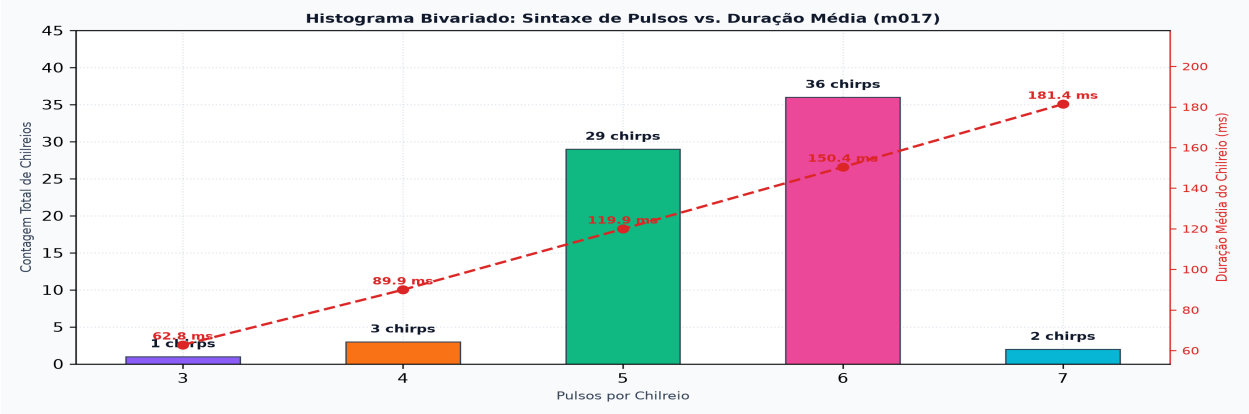


Figura 3.2: Histograma Bivariado de Sintaxe Temporal (Distribuição conjunta de Pulsos por Chilreio vs. Duração do Chilreio em ms).

[GLOSSÁRIO TÉCNICO E CIENTÍFICO DA SEÇÃO §3.0 - §3.8]

- **[G-13] Gerador Central de Padrões (CPG):** Rede interneuronal torácica que comanda os ciclos motores rítmicos de estridulação em ortópteros.
- **[G-14] Envoltória de Hilbert:** Curva analítica instantânea obtida a partir da amplitude complexa do sinal, refletindo a energia mecânica do pulso.
- **[G-15] Duração FWHM (-6 dB):** Largura temporal do pulso medida a 50% da amplitude máxima linear (Full Width at Half Maximum).
- **[G-16] Duty Cycle (Ciclo de Trabalho):** Percentual da duração do pulso em relação ao período total do ciclo acústico (IPI).
- **[G-17] Filtro de Coerência Rítmica O(N):** Algoritmo de varredura linear que expurga transientes que violam o ritmo neuromuscular nas bordas do chilreio.
- **[G-18] Portão ICI (ICI Gate):** Critério refratário que consolida sub-agrupamentos separados por pausas menores que 70% da mediana global.

4.0. CAPÍTULO 4: APRENDIZADO DE MÁQUINA HÍBRIDO E ENGENHARIA DE ATRIBUTOS

A bioacústica de campo frequentemente enfrenta cenários de polifonia complexa, onde múltiplos insetos cantam simultaneamente em planos acústicos distintos. O módulo de Inteligência Artificial do Crinômetro (PulseLearner) combina modelagem probabilística não-supervisionada, classificação supervisionada por gradiente impulsionado, indução de regras contrastivas e aprendizado ativo com feedback humano incremental.

4.1. Acústica Ecológica: Segregação do Canto Focal em Coros Polifônicos e Invariância a Ganho

[BLOCO A: FUNDAMENTOS FÍSICOS E BIOACÚSTICOS]

Em ecossistemas tropicais, machos de grilos cantam em agregados espaciais densos. O microfone capta o macho focal (a poucos centímetros) sobreposto a dezenas de machos conspecíficos e heterospecíficos a distâncias variadas (1 a 20 metros). A propagação da onda sonora sofre duas atenuações fundamentais: *(1)* **Divergência geométrica esférica:** a pressão acústica decai com o inverso da distância (p ~ 1/r); *(2)* **Absorção atmosférica molecular:** dissipação viscosa e relaxação molecular do ar que atenuam exponencialmente as altas frequências em função da umidade relativa e temperatura. Ademais, gravações de campo feitas por diferentes pesquisadores exibem ganhos analógicos arbitrários. Qualquer classificador que utilize amplitude absoluta é inútil em ecologia bioacústica.

[BLOCO B: FORMULAÇÃO MATEMÁTICA E ESTATÍSTICA]

Para garantir transferibilidade inter-gravações, o Crinômetro formula seus descritores com **invariância estrita a ganho linear**. Para qualquer fator multiplicativo de ganho k > 0 aplicado ao sinal bruto x'(t) = k · x(t), os atributos baseados em razões de amplitude local ou relações de energia satisfazem:

$$f_i(k \cdot x) = [\, k \cdot \hat{E}[p] \,] / [\, med(k \cdot \hat{E}_{local}) \,] = \hat{E}[p] / med(\hat{E}_{local}) = f_i(x)$$

[BLOCO C: ARQUITETURA E IMPLEMENTAÇÃO DE CÓDIGO (v4.5.0)]

```
# Normalização local com invariância a ganho em core/learner.py
peak_amp = float(env_signal[peak_idx])
med_amp = float(np.median(env_segment)) + 1e-6
peak_amp_rel = peak_amp / med_amp  # Invariante a qualquer ganho k
local_mad = float(np.median(np.abs(border_env - np.median(border_env)))) + 1e-6
local_snr_db = float(20.0 * np.log10(peak_amp / (1.4826 * local_mad)))
```

4.2. O Espaço Vetorial de 20 Descritores Bioacústicos Normalizados

Para cada pulso candidato, o PulseLearner constrói um vetor de características $f(p_i)$ projetado no espaço vetorial $\mathbf{R}^{20}$, extraído em janelas centradas no ápice do envelope (Tabela 4.1):

ID	Nome do Atributo	Formulação Matemática Rigorosa	Interpretação Bioacústica Mecânica
1	peak_amp_rel	Ê[p] / (mediana(Ê_±30ms) + 1e-6)	Razão de emergência do pulso focal sobre o ruído da vizinhança local.
2	peak_width_s	w_0.50 / rate	Duração FWHM a -6 dB em segundos (caractere biomecânico diagnóstico).
3	peak_width_75_s	w_0.25 / rate	Largura do topo do pulso a 75% da altura (agudeza do impacto tegminal).
4	local_snr_db	20 log10(Ê[p] / (1.4826 MAD(borda) + 1e-6))	Relação Sinal-Ruído local robusta imune a outliers na vizinhança.
5	energy_rel	mean(Ê_±30ms²) / (Ê[p]² + 1e-9)	Concentração relativa da densidade de energia acústica da janela.
6	std_amp_rel	std(Ê_±30ms) / (Ê[p] + 1e-6)	Grau de variabilidade estocástica da amplitude ao redor do pulso.
7	prominence_ratio	(Ê[p] - min(vales)) / (Ê[p] + 1e-6)	Proeminência topográfica sobre vales adjacentes (isolamento da crista).
8	peak_sharpness_rel	(2Ê[p] - Ê[p-δ] - Ê[p+δ]) / (Ê[p] + 1e-6)	Curvatura de 2ª ordem no ápice (taxa de impacto dente-a-dente).
9	crest_factor	Ê[p] / (RMS_±30ms + 1e-6)	Fator de crista temporal (discrimina cliques de transientes suaves).

ID	Nome do Atributo	Formulação Matemática Rigorosa	Interpretação Bioacústica Mecânica
10	skewness_local	(mean(esq) - mean(dir)) / (Ê[p] + 1e-6)	Assimetria temporal do envelope (ataque tegminal vs decaimento ressonante).
11	local_density_rel	mean(Ê_±10ms) / (mean(Ê_±30ms) + 1e-6)	Taxa de confinamento de energia no núcleo temporal de 10 ms.
12	autocorr_1ms	∑ e[m] e[m-τ] / (∑ e[m]² + 1e-9)	Coeficiente de autocorrelação com atraso de 1 ms (periodicidade residual).
13	band_energy_ratio	E_±10ms / (E_±30ms + 1e-9)	Foco energético da sílaba focal em relação ao contexto temporal largo.
14	rise_slope_rel	(Ê[p] - min(esq)) / (Δt_subida · Ê[p])	Velocidade mecânica de fechamento tegminal durante o ataque sonoro.
15	spectral_centroid	∑ f |X[f]| / (∑ |X[f]| + 1e-9)	Centro de gravidade espectral (Hz); detecta atenuação por distância.
16	spectral_rolloff_85	Frequência onde energia atinge 85%	Limite superior da irradiação acústica da harpa/espelho (Hz).
17	attack_time_10_90_ms	(t_90% - t_10%) × 1000	Tempo de subida entre 10% e 90% da altura via interpolação linear (ms).
18	local_hnr_db	10 log10(E_portadora / (E_fora + 1e-9))	Razão Harmônico-Ruído local na janela de ±150 Hz em torno de fc.
19	multiscale_energy	RMS_15ms / (RMS_80ms + 1e-9)	Densidade de potência multiescala (pulso isolado vs coro polifônico).
20	prior_p_focal	P(Cluster Focal | GMM Bimodal)	Probabilidade posterior não-supervisionada gerada pelo modelo GMM.

[BLOCO C: ARQUITETURA E IMPLEMENTAÇÃO DE CÓDIGO (v4.5.0)]

```
# Trecho de core/learner.py (PulseLearner.extract_features_for_peak)
feat = np.asarray([
    peak_amp_rel, peak_width_s, peak_width_75_s, local_snr_db, energy_rel,
    std_amp_rel, prominence_ratio, peak_sharpness_rel, crest_factor, skewness_local,
    local_density_rel, autocorr_1ms, band_energy_ratio, rise_slope_rel,
    spectral_centroid, spectral_rolloff_85, attack_time_10_90_ms, local_hnr_db,
    multiscale_energy_ratio
], dtype=float)
return np.nan_to_num(feat, nan=0.0, posinf=10000.0, neginf=-10000.0)
```

4.3. Classificador Não-Supervisionado: Modelo de Mistura de Gaussianas (GMM Bimodal)

[BLOCO A: FUNDAMENTOS FÍSICOS E BIOACÚSTICOS]

Antes que o taxonomista realize qualquer anotação manual, o sistema precisa fornecer uma primeira estimativa confiável de quais pulsos pertencem ao espécime focal mais próximo e quais decorrem de insetos distantes ou ruídos. O Modelo de Mistura de Gaussianas (GMM) modela essa partição bimodal intrínseca de forma puramente não-supervisionada.

[BLOCO B: FORMULAÇÃO MATEMÁTICA E ESTATÍSTICA]

O GMM opera em um subespaço bidimensional padronizado Z-score: z = [log(amplitude), centroide_espectral]. Ajustam-se duas componentes gaussianas multivariadas via algoritmo de Expectation-Maximization (EM):

$$P(z) = \sum_{k=1}^{2} \pi_k \, N(z \mid \mu_k, \Sigma_k), \qquad c_{focal} = \operatorname{argmax}_k \left(\mu_{k,\,amp} + 0.5\, \mu_{k,\,centroide} \right)$$

O cluster focal é eleito pela soma ponderada de sua média de amplitude e centroide espectral (pulsos próximos sofrem menos atenuação atmosférica e possuem centroide mais alto). Instancia-se com n_components=2, matriz de covariância covariance_type='diag' ou 'full' e n_init=5 inicializações. Salvaguarda: se o áudio contiver N < 10 pulsos, o algoritmo assume ausência de coro e atribui P(focal) ≡ 1.0.

[BLOCO C: ARQUITETURA E IMPLEMENTAÇÃO DE CÓDIGO (v4.5.0)]

```
# Trecho de core/learner.py (PulseLearner.compute_unsupervised_focal_priors)
if n < 10 or GaussianMixture is None:
    return np.ones(n, dtype=float)  # Salvaguarda para amostras escassas
F_norm = (F - np.mean(F, axis=0)) / (np.std(F, axis=0) + 1e-6)
gmm = GaussianMixture(n_components=2, covariance_type='diag', random_state=42, max_iter=100)
gmm.fit(F_norm)
c_focal = int(np.argmax(gmm.means_[:, 0] + 0.5 * gmm.means_[:, 1]))
probs = gmm.predict_proba(F_norm)[:, c_focal]
return np.clip(probs, 0.0, 1.0)
```

4.4. Classificador Supervisionado (HistGradientBoosting) e Fallback para RandomForest

[BLOCO A: FUNDAMENTOS FÍSICOS E BIOACÚSTICOS]

Quando o taxonomista interage com a interface (adicionando sílabas perdidas ou excluindo ruídos), o sistema assimila esse conhecimento biológico humano e calibra um ensemble supervisionado de alta velocidade para propagar a diagnose correta ao restante da gravação.

[BLOCO B: FORMULAÇÃO MATEMÁTICA E ESTATÍSTICA]

O classificador primário é o **HistGradientBoostingClassifier** do scikit-learn. Sua principal vantagem matemática é a discretização dos atributos contínuos em histogramas inteiros de 256 bins inteiros de 8 bits (uint8). Isso reduz a complexidade de ordenação de divisões de nós de **O(N log N)** (GBDT clássico) para **O(N)**, permitindo o retreinamento do modelo em menos de 100 ms durante o clique do usuário.
Hiperparâmetros configurados:
• *Poucas amostras (N < 30):* max_iter=50, max_depth=3, min_samples_leaf=2, learning_rate=0.08, l2_regularization=2.0 (regularização forte contra overfitting);
• *Amostras abundantes (N ≥ 30):* max_iter=80, max_depth=4, min_samples_leaf=5, learning_rate=0.08, l2_regularization=1.0, class_weight='balanced'.
Caso o ambiente Python não possua os módulos C do histogram boosting, o sistema ativa fallback transparente para RandomForestClassifier(n_estimators=80, max_depth=5, min_samples_split=6, class_weight='balanced', n_jobs=-1).

[BLOCO C: ARQUITETURA E IMPLEMENTAÇÃO DE CÓDIGO (v4.5.0)]

```
# Trecho de core/learner.py (PulseLearner._build_model)
if HistGradientBoostingClassifier is not None:
    if n_samples > 0 and n_samples < 30:
        return HistGradientBoostingClassifier(
            max_iter=50, max_depth=3, min_samples_leaf=2,
            learning_rate=0.08, l2_regularization=2.0,
            random_state=42, class_weight='balanced'
        )
    return HistGradientBoostingClassifier(
        max_iter=80, max_depth=4, min_samples_leaf=5,
        learning_rate=0.08, l2_regularization=1.0,
        random_state=42, class_weight='balanced'
    )
elif RandomForestClassifier is not None:
    return RandomForestClassifier(n_estimators=80, max_depth=5, class_weight='balanced', n_jobs=-1)
```

4.5. Mineração de Negativos Difíceis (Hard Negative Mining) e Poda Contrastiva

[BLOCO A: FUNDAMENTOS FÍSICOS E BIOACÚSTICOS]

Sons transientes espúrios como gotas de orvalho caindo no microfone ou cliques de insetos predadores podem exibir alta amplitude e enganar classificadores estatísticos convencionais. Quando o pesquisador clica para remover um falso positivo, o Crinômetro analisa o vetor de atributos desse negativo específico e induz regras determinísticas rígidas.

[BLOCO B: FORMULAÇÃO MATEMÁTICA E ESTATÍSTICA]

Para métricas taxonômicas críticas (centroide, fator de crista, largura FWHM, tempo de subida 10-90%, roll-off e HNR), o algoritmo compara a distribuição dos pulsos confirmados (Válidos) contra os falsos positivos explícitos (FP). Se houver separação perfeita com gap positivo g > 0:

• Caso 1 (min(V) > max(FP)): Threshold = max(FP) + 0.30 × g (regra: descartar se f_i < Threshold);

• Caso 2 (max(V) < min(FP)): Threshold = min(FP) - 0.30 × g (regra: descartar se f_i > Threshold).

A margem de 30% evita descartes de sílabas limítrofes. As regras atuam como um filtro prévio determinístico em tempo **O(1)**. Salvaguarda: se as regras podarem mais de 75% dos candidatos de uma nova gravação, a poda rígida é suspensa, restringindo-se apenas aos pulsos de baixíssima confiança.

[BLOCO C: ARQUITETURA E IMPLEMENTAÇÃO DE CÓDIGO (v4.5.0)]

```
# Trecho de core/learner.py (PulseLearner.induce_pruning_rules)
min_val, max_val = float(np.min(v_val)), float(np.max(v_val))
min_fp, max_fp = float(np.min(v_fp)), float(np.max(v_fp))
if min_val > max_fp:
    gap = min_val - max_fp
    threshold = max_fp + 0.30 * gap  # Margem de segurança de 30%
    new_rules.append({'feature': feat_name, 'op': '<', 'threshold': float(threshold)})
elif max_val < min_fp:
    gap = min_fp - max_val
    threshold = min_fp - 0.30 * gap
    new_rules.append({'feature': feat_name, 'op': '>', 'threshold': float(threshold)})
```

4.6. Pontuação Híbrida Ponderada e Resgate Contextual (Contextual Gating)

[BLOCO A: FUNDAMENTOS FÍSICOS E BIOACÚSTICOS]

A primeira e a última sílaba de um chilreio frequentemente possuem energia significativamente inferior às sílabas centrais devido à inércia do movimento de fechamento da asa. Um classificador estático descartaria essas sílabas de borda. O mecanismo de Resgate Contextual recupera esses pulsos periféricos se eles exibirem coerência sintática com o ritmo do chilreio.

[BLOCO B: FORMULAÇÃO MATEMÁTICA E ESTATÍSTICA]

A decisão final combina a probabilidade a posteriori do ensemble supervisionado (peso 65%, refletindo a curadoria humana) com o prior não-supervisionado do GMM (peso 35%):

$$Score(p_i) = 0{,}65 \cdot P_{sup}(y=1 \mid f_i) + 0{,}35 \cdot P_{GMM}(Focal \mid z_i)$$

Pulsos com Score ≥ focal_sensitivity (padrão 0.60) são aprovados diretamente. Em seguida, executa-se o **Contextual Gating** iterativo (até 5 passagens de propagação temporal): qualquer pulso inicialmente descartado que esteja pareado com um pulso vizinho aprovado por intervalo biológico gap_min · 0.85 ≤ Δt ≤ gap_max · 1.15 e cuja amplitude atinja pelo menos 20% do vizinho é resgatado e reintegrado ao conjunto focal.

[BLOCO C: ARQUITETURA E IMPLEMENTAÇÃO DE CÓDIGO (v4.5.0)]

```
# Trecho de core/learner.py (PulseLearner.filter_peaks - Score Híbrido & Gating)
combined_score = 0.65 * sup_probs + 0.35 * focal_priors
kept_mask = (combined_score >= focal_sensitivity) & (amps >= 0.04)

# Resgate Contextual iterativo por coerência de IPI fisiológico
for _ in range(5):
    changed = False
    for i in range(n):
        if not kept_mask[i] and combined_score[i] >= 0.35:
            has_match = (i > 0 and kept_mask[i-1] and gap_min_s*0.85 <= (t[i]-t[i-1]) <= gap_max_s*1.15)
            if has_match:
                kept_mask[i] = True
                changed = True
    if not changed:
        break
```

4.7. Ciclo de Feedback do Aprendizado Ativo em 5 Passos (subpassos a até g)

O aprendizado ativo do Crinômetro segue o paradigma de pool-based active learning em 5 passos formais:
Passo 1: Detecção Inicial DSP: O pipeline processa o áudio bruto e gera candidatos preliminares baseados nas heurísticas de Hilbert e limiares adaptativos.
Passo 2: Classificação Automática: O PulseLearner aplica o GMM e o classificador treinado, segregando cantos focais de ruídos.
Passo 3: Revisão Humana Interativa: O taxonomista inspeciona o oscilograma/espectrograma. Marcadores incorretos são removidos (clique direito); sílabas omitidas são adicionadas (clique esquerdo).
Passo 4: Salvamento e Atualização Transacional com 7 Subpassos Obrigatórios:

a) Extração de 20 features de todos os pulsos validados (labels positivas y=1);
b) Extração de features dos pulsos explicitamente removidos pelo usuário (labels negativas y=0);
c) Mineração automática de negativos adicionais (silêncios e vales de baixa energia na gravação);
d) Indução de regras de poda contrastivas (Hard Negatives) com margem de 30%;
e) Acumulação no buffer circular histórico limitado a **5000 amostras** (preservando generalização);
f) Retreinamento do classificador com o dataset balanceado e regularização L2;
g) Serialização atômica segura via skops.io em modelo_treinado.skops.

Passo 5: Adaptação Paramétrica: Calibração automática dos limiares de DSP (proeminência e tolerância) às características estatísticas da espécie anotada.

[BLOCO C: ARQUITETURA E IMPLEMENTAÇÃO DE CÓDIGO (v4.5.0)]

```
# Trecho de core/learner.py (PulseLearner.update_from_corrections)
stacked = np.column_stack((X, y))
_, first_idx = np.unique(stacked, axis=0, return_index=True)
stacked = stacked[np.sort(first_idx)]
self.training_features = stacked[:, :-1]
self.training_labels = stacked[:, -1].astype(int)

# Limite do buffer circular histórico de 5000 amostras
_MAX_SAMPLES = 5000
if len(self.training_features) > _MAX_SAMPLES:
    self.training_features = self.training_features[-_MAX_SAMPLES:]
    self.training_labels = self.training_labels[-_MAX_SAMPLES:]
```

4.8. Serialização Segura via `skops` e Desacoplamento de Locks de Concorrência

[BLOCO A: FUNDAMENTOS FÍSICOS E BIOACÚSTICOS]

A colaboração científica entre instituições e herbários/museus exige o compartilhamento de arquivos de modelos bioacústicos treinados para identificação de espécies crípticas. O formato clássico .pkl (Pickle) executa bytecode arbitrário durante o carregamento, representando uma grave vulnerabilidade de cibersegurança (RCE - Remote Code Execution). O Crinômetro v4.5.0 aboliu totalmente o pickle em prol do formato skops auditado.

[BLOCO B: FORMULAÇÃO MATEMÁTICA E ESTATÍSTICA]

O protocolo skops.io armazena a estrutura do modelo em árvore sintática tipada (AST) sem permitir instruções executáveis de baixo nível. Durante o carregamento, aplica-se uma lista branca estrita de tipos confiáveis: HistGradientBoostingClassifier, RandomForestClassifier, np.ndarray, TreePredictor e Tree. Qualquer objeto fora da lista branca aborta a leitura imediatamente.
Desacoplamento de Concorrência: Para eliminar deadlocks clássicos do tipo AB-BA (onde duas threads competem pelo Lock do modelo ML e pelo Lock de gravação do JSON de configuração em ordens opostas), os locks _model_lock e CONFIG_LOCK jamais são aninhados; a serialização em memória ocorre sob o lock do modelo, liberando-o antes da aquisição do lock de gravação em disco.

[BLOCO C: ARQUITETURA E IMPLEMENTAÇÃO DE CÓDIGO (v4.5.0)]

```
# Trecho de core/learner.py (Serialização skops e Desacoplamento de Locks)
def save_to_config(self):
    with self._model_lock:
        if self.model is None:
            return False
        serialized = self.serialize()  # Serializa sob _model_lock
        rules = list(getattr(self, 'pruning_rules', []))
    # _model_lock LIBERADO antes de adquirir CONFIG_LOCK (Evita Deadlock AB-BA)
    with CONFIG_LOCK:
        config['pulse_learner'] = serialized
        atomic_write_json(self.config_path, config)

# Carregamento auditado com lista branca em skops.io
with open(target_path, 'rb') as f:
    payload = skio.load(f, trusted=[
        HistGradientBoostingClassifier, RandomForestClassifier,
        np.ndarray, np.dtype, 'sklearn.tree._tree.Tree'
    ])
```

[GLOSSÁRIO TÉCNICO E CIENTÍFICO DA SEÇÃO §4.0 - §4.8]

- **[G-19] Invariância a Ganho Linear:** Propriedade de descritores normalizados que mantêm seus valores idênticos mesmo se o ganho de captação for alterado.
- **[G-20] Gaussian Mixture Model (GMM):** Modelo probabilístico generativo que ajusta componentes gaussianas multivariadas para identificar subpopulações de pulsos.
- **[G-21] HistGradientBoosting:** Ensemble de árvores de decisão impulsionadas por gradiente com discretização de atributos em histogramas de tempo O(N).
- **[G-22] Contextual Gating:** Mecanismo bioacústico iterativo que resgata sílabas periféricas débeis desde que alinhadas ao ritmo estrito do chilreio.
- **[G-23] Serialização skops:** Formato seguro de armazenamento de modelos scikit-learn baseado em árvore de tipos, imune a injeções de bytecode executável.
- **[G-24] Desacoplamento de Locks AB-BA:** Padrão de concorrência que impede que duas threads tentem adquirir travas mútuas em ordem invertida, erradicando travamentos.

5.0. CAPÍTULO 5: COMPUTAÇÃO GRÁFICA, ACELERAÇÃO LOD E INTERFACE REATIVA

A visualização de registros bioacústicos de campo impõe desafios severos à computação gráfica. Gravações contínuas de 5 a 30 minutos contêm de 10 a 50 milhões de pontos amostrais. A renderização simultânea de todos os vértices sobrecarrega o backend de rasterização (Agg) do Matplotlib, degradando a taxa de quadros para níveis inoperantes (< 2 FPS). O Crinômetro v4.5.0 introduz uma arquitetura de motores gráficos dedicados (Level-of-Detail - LoD) ancorada em garantias matemáticas de preservação de sinal.

5.1. Teorema de Preservação de Transientes por Decimação Min-Max LoD

[BLOCO A: FUNDAMENTOS FÍSICOS E BIOACÚSTICOS]

A abordagem ingênua para acelerar a renderização de áudios densos consiste na subamostragem por passo fixo (*stride*: pegar uma amostra a cada K pontos). Na bioacústica de Orthoptera, essa técnica é desastrosa: pulsos estridulatórios curtos de 3 a 10 ms frequentemente 'caem nos vãos' entre as amostras selecionadas, desaparecendo completamente da tela (fenômeno conhecido como *Aliasing Visual de Amostragem*). O pesquisador tem a falsa ilusão de que o grilo parou de cantar.

[BLOCO B: FORMULAÇÃO MATEMÁTICA E ESTATÍSTICA]

O motor HighPerfLineEngine implementa a **Decimação Min-Max LoD**. O domínio visível da tela $[t_{start}, t_{end}]$ (contendo N_{vis} amostras) é particionado em B blocos contíguos disjuntos $I_b = [t_{b,0}, t_{b,K-1}]$, de tamanho $K = \text{floor}(N_{vis} / B)$, onde B varia entre 300 e 4000 de acordo com o modo de interação. Cada bloco temporal é mapeado em exatamente dois vértices ordenados no tempo:

$$y_{LoD}[2b] = \min_{n \text{ em } I_b} y[n], \quad y_{LoD}[2b+1] = \max_{n \text{ em } I_b} y[n], \quad \text{Complexidade: } O(N_{vis}) \rightarrow O(B)$$

Enunciado e Prova Formal do Teorema de Preservação:

Teorema: Seja p^* o instante temporal de um pulso biológico de amplitude máxima local $y[p^*] = \max_{n \text{ em } V(p^*)} y[n]$. Para qualquer número de blocos $B \geq 1$, a envolvente visual renderizada pelo algoritmo Min-Max satisfaz $\max_{n \text{ em } I_{b^*}} y_{LoD} = y[p^*]$, onde I_{b^*} é o bloco que contém p^*.

Prova: Pela definição de supremo sobre subconjuntos, sendo p^* pertencente a I_{b^*}, temos que $\max_{n \text{ em } I_{b^*}} y[n] \geq y[p^*]$. Como p^* é o pico máximo no intervalo, $\max_{n \text{ em } I_{b^*}} y[n] \equiv y[p^*]$. Pelo passo construtivo do algoritmo, o vértice superior do bloco é atribuído como $y_{max} = \max_{n \text{ em } I_{b^*}} y[n] = y[p^*]$. Portanto, o valor de pico exibido no pixel da tela coincide com precisão exata com a amplitude física do sinal analógico original. Nenhum transiente estridulatório é atenuado ou ocultado, independentemente da taxa de zoom. Q.E.D.

[BLOCO C: ARQUITETURA E IMPLEMENTAÇÃO DE CÓDIGO (v4.5.0)]

```
# Trecho de core/engines.py (HighPerfLineEngine._get_envelope)
def _get_envelope(self, t, y, max_points=4000):
    if len(y) <= max_points:
        return t, y
    num_blocks = max_points // 2
    block_size = len(y) // num_blocks
    y_trunc = y[:block_size * num_blocks].reshape(num_blocks, block_size)
    t_trunc = t[:block_size * num_blocks].reshape(num_blocks, block_size)
    idx_min, idx_max = y_trunc.argmin(axis=1), y_trunc.argmax(axis=1)
    row_idx = np.arange(num_blocks)
    t_comb = np.column_stack((t_trunc[row_idx, idx_min], t_trunc[row_idx, idx_max]))
    y_comb = np.column_stack((y_trunc[row_idx, idx_min], y_trunc[row_idx, idx_max]))
    sort_idx = np.argsort(t_comb, axis=1)
    t_env = np.take_along_axis(t_comb, sort_idx, axis=1).flatten()
    y_env = np.take_along_axis(y_comb, sort_idx, axis=1).flatten()
    return t_env, y_env
```

5.2. Arquitetura das Engines Vetorizadas: HighPerfLineEngine e HighPerfSpectrogramEngine

[BLOCO A: FUNDAMENTOS FÍSICOS E BIOACÚSTICOS]

A interface gráfica deve oferecer resposta instantânea durante o arraste e rolagem do áudio. Para conciliar velocidade com riqueza de detalhes, os motores adotam uma estratégia de renderização em dois estágios.

[BLOCO B: FORMULAÇÃO MATEMÁTICA E ESTATÍSTICA]

O sistema opera em dois níveis de detalhe acionados dinamicamente:
(1) Estágio Interativo Rápido (render_interactive): Quando o usuário move a barra de rolagem ou arrasta o mouse, a decimação opera com B = 300 a 500 vértices para a forma de onda e fatiamento temporal com passo lod_step = viewport_width // 200 para o espectrograma. A latência de quadro cai para menos de **12 ms** (sustentando > 60 FPS).
(2) Estágio de Alto Detalhe (render_high_detail): Ao cessar a interação por mais de 120-150 ms (controlado por um QTimer com debounce automático), o motor recalcula o envelope com B = 4000 pontos e re-renderiza o espectrograma em resolução nativa completa pixel a pixel.
O fatiamento do viewport opera em tempo **O(1)** via busca binária rápida np.searchsorted nos vetores de tempo e frequência.

[BLOCO C: ARQUITETURA E IMPLEMENTAÇÃO DE CÓDIGO (v4.5.0)]

```
# Trecho de core/engines.py (Estratégia de 2 Estágios e Viewport Slice O(1))
def render_interactive(self, xmin, xmax, is_sync=False):
    self.debounce_timer.stop()
    t0, t1 = self.get_viewport_slice(xmin, xmax)
    pts = 300 if is_sync else 500  # Modo ultrarrápido a 60 FPS
    t_env, y_env = self._get_envelope(self.time_sec[t0:t1], self.data[t0:t1], max_points=pts)
    self._update_plot(t_env, y_env)
    self.debounce_timer.start(150 if is_sync else 120)  # Agenda render_high_detail
```

5.3. Interatividade a 60 FPS: Background Blitting Coalescente e Poda Espacial O(log M) no Hover

[BLOCO A: FUNDAMENTOS FÍSICOS E BIOACÚSTICOS]

Ao inspecionar gravações, o pesquisador move o mouse sobre o oscilograma para ler instantaneamente o tempo, amplitude e diagnóstico de cada sílaba. Se cada movimento do mouse (gerado a 125-1000 Hz pelo sistema operacional) provocasse o redesenho completo de todos os eixos, o sistema congelaria.

[BLOCO B: FORMULAÇÃO MATEMÁTICA E ESTATÍSTICA]

Para atingir 60 FPS estáveis, o NavigationController adota restauração por *Bit-Block Transfer* (BitBlt). Após o desenho dos eixos estáticos e curvas principais, a matriz de pixels do canvas é armazenada em buffer na memória RAM via copy_from_bbox(ax.bbox). Durante o evento de movimento do mouse (on_motion), o fundo original é restaurado instantaneamente via restore_region(bg) em tempo de cópia de memória RAM (< 1.5 ms), redesenhando-se apenas as linhas de retícula e o marcador dinâmico via draw_artist().
Poda Espacial O(log M) no Hover: A busca pelo pulso sob o cursor restringe-se a uma vizinhança de ±30 pixels de tela. Convertendo a coordenada em tempo x_{cur}, o algoritmo aplica **busca binária dicotômica (np.searchsorted)** sobre o vetor de tempos ordenados dos M picos validados, reduzindo a complexidade de busca de O(M) para O(log M).

[BLOCO C: ARQUITETURA E IMPLEMENTAÇÃO DE CÓDIGO (v4.5.0)]

```
# Trecho de ui/controllers/navigation_controller.py (Blitting e Poda O(log M))
# Restaura fundo pré-renderizado em < 1.5 ms
canvas.restore_region(self._bg_cache[ax])
# Poda dicotômica restrita à vizinhança temporal de ±30 pixels
t_mouse = event.xdata
idx = np.searchsorted(peak_times, t_mouse)
candidates = []
for k in [idx - 1, idx, idx + 1]:
    if 0 <= k < len(peak_times) and abs(peak_times[k] - t_mouse) <= dt_tol_30px:
        candidates.append(peaks[k])
# Redesenha apenas os artistas dinâmicos e atualiza a GPU
ax.draw_artist(cursor_line)
canvas.blit(ax.bbox)
```

5.4. Controle Transacional de Visualização e Renderização Isolada de Eixos (NavigationController)

[BLOCO A: FUNDAMENTOS FÍSICOS E BIOACÚSTICOS]

A correlação simultânea da forma de onda no tempo e da energia no espectrograma exige que o eixo temporal permaneça estritamente sincronizado entre todos os subplots. Uma ampliação no oscilograma deve refletir imediatamente no espectrograma sem disparar ciclos infinitos de eventos de callback mútuos.

[BLOCO B: FORMULAÇÃO MATEMÁTICA E ESTATÍSTICA]

O NavigationController implementa uma trava transacional reentrante baseada na flag self._updating_axes. Quando um evento de zoom/pan é detectado em qualquer dos eixos, a flag bloqueia a propagação reflexa. Os limites temporais [x_{min}, x_{max}] são propagados de forma unidirecional, enquanto os eixos verticais de amplitude [-1.0, +1.0] e frequência [0, f_{Nyq}] mantêm isolamento geométrico total.

[BLOCO C: ARQUITETURA E IMPLEMENTAÇÃO DE CÓDIGO (v4.5.0)]

```
# Trecho de ui/controllers/navigation_controller.py (Sincronização Unidirecional de Eixos)
def on_xlims_changed(self, ax_source):
    if self._updating_axes:
        return
    self._updating_axes = True
    try:
        new_xlim = ax_source.get_xlim()
        for target_ax in self.managed_axes:
            if target_ax is not ax_source:
                target_ax.set_xlim(new_xlim, emit=False)
        self.request_lod_update(new_xlim)
    finally:
        self._updating_axes = False
```

5.5. Reavaliação Espectral em Tempo Real com Debounce de 25 ms

[BLOCO A: FUNDAMENTOS FÍSICOS E BIOACÚSTICOS]

Durante a exploração de um espécime novo, o taxonomista precisa ajustar interativamente a tolerância espectral (por exemplo, abrindo de ±300 Hz para ±800 Hz) para avaliar se sílabas limítrofes são incluídas. Recalcular a STFT e a envoltória de Hilbert a cada movimento do slider tornaria a ferramenta inutilizável.

[BLOCO B: FORMULAÇÃO MATEMÁTICA E ESTATÍSTICA]

O método CricketAnalyzer.reevaluate_carrier_tolerance desacopla os dados analíticos pré-computados (valid_peaks_stage2, dom_freqs, matriz de potências). A reclassificação e o agrupamento sintático executam diretamente na memória RAM em menos de **5 ms**. O sinal de interface é amortecido por um temporizador de **debounce de 25 ms** (QTimer.singleShot(25, callback)), coalescendo múltiplos micro-eventos do mouse.

[BLOCO C: ARQUITETURA E IMPLEMENTAÇÃO DE CÓDIGO (v4.5.0)]

```
# Trecho de core/analyzer.py (CricketAnalyzer.reevaluate_carrier_tolerance)
@staticmethod
def reevaluate_carrier_tolerance(valid_peaks_stage2, carrier_freq, new_tol_hz,
                                 rate, t_spec, dom_freqs, env, params, **kwargs):
    # Reclassificação instantânea na RAM sem recalcular transformadas (< 5 ms)
    valid_peaks_stage3 = []
    for p in valid_peaks_stage2:
        f_p = float(np.interp(p / rate, t_spec, dom_freqs))
        if abs(f_p - carrier_freq) <= new_tol_hz:
            valid_peaks_stage3.append(p)
    peaks = np.asarray(sorted(valid_peaks_stage3), dtype=int)
    return CricketAnalyzer.regroup_chirps(peaks, params, rate, env)
```

5.6. Geração Pura de Figuras Científicas Vetoriais e Raster HD (FigureFactory)

[BLOCO A: FUNDAMENTOS FÍSICOS E BIOACÚSTICOS]

A publicação de novas espécies exige ilustrações vetoriais e gráficos rasterizados em altíssima resolução (300 a 600 DPI para periódicos internacionais). A geração não pode sofrer influência da resolução do monitor ou do tema visual ativo na interface gráfica.

[BLOCO B: FORMULAÇÃO MATEMÁTICA E ESTATÍSTICA]

O módulo FigureFactory adota a API pura orientada a objetos do Matplotlib (FigureCanvasAgg), com banimento explícito do submódulo global com estado matplotlib.pyplot. Isso erradica o vazamento crônico de memória associado a instâncias não-fechadas de figuras no Python. A exportação suporta renderização vetorial exata em PDF/SVG e rasterização em PNG/TIFF.

[BLOCO C: ARQUITETURA E IMPLEMENTAÇÃO DE CÓDIGO (v4.5.0)]

```
# Trecho de ui/figure_factory.py (Exportação Pura via FigureCanvasAgg)
from matplotlib.figure import Figure
from matplotlib.backends.backend_agg import FigureCanvasAgg

def create_publication_figure(data_dict, export_config):
    fig = Figure(figsize=(export_config.width_in, export_config.height_in), dpi=export_config.dpi)
    canvas = FigureCanvasAgg(fig)
    ax = fig.add_subplot(111)
    ax.plot(data_dict['time'], data_dict['amplitude'], color='#1E293B', lw=0.6)
    fig.tight_layout()
    canvas.print_figure(export_config.filepath, dpi=export_config.dpi)
    fig.clf()  # Liberação compulsória de memória RAM
```

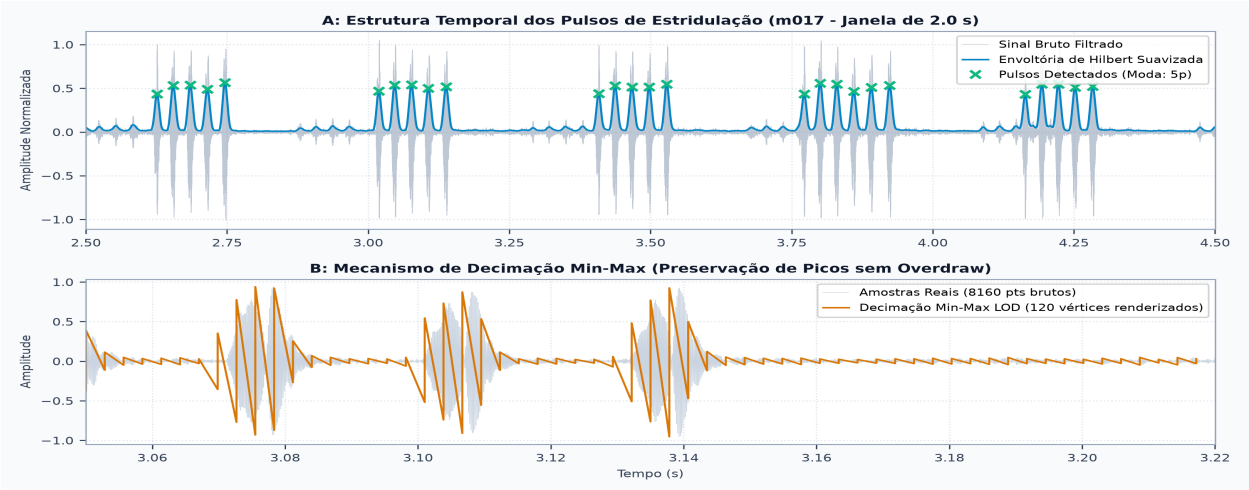


Figura 5.1: Arquitetura do Mecanismo Min-Max LoD: Sinal Completo (Topo) vs. Detalhe Decimado sem Perda de Transientes de Pico (Base).

[GLOSSÁRIO TÉCNICO E CIENTÍFICO DA SEÇÃO §5.0 - §5.6]

- **[G-25] Aliasing Visual de Amostragem:** Desaparecimento de picos estreitos na tela quando um sinal de áudio denso é sub-amostrado por passos fixos.
- **[G-26] Decimação Min-Max LoD:** Algoritmo de redução que extrai o valor mínimo e máximo de cada balde visual, garantindo a visualização da amplitude total.
- **[G-27] Raster Blitting (BitBlt):** Operação de memória que restaura regiões de pixels pré-renderizadas na RAM em tempo O(1), permitindo animação a 60 FPS.
- **[G-28] Poda Espacial O(log M):** Busca binária dicotômica de eventos no eixo do tempo restrita a uma vizinhança de pixels do cursor do mouse.
- **[G-29] Coalescência de Blits (Debounce):** Agrupamento de múltiplos eventos rápidos de interface em um único ciclo de redesenho para evitar sobrecarga de GPU/CPU.
- **[G-30] FigureFactory Pura:** Gerador isolado de gráficos baseado em API pura do Matplotlib (FigureCanvasAgg), imune a vazamentos de memória de pyplot.

6.0. CAPÍTULO 6: PROTOCOLOS EXPERIMENTAIS, CONSOLIDAÇÃO E ÍNDICE REMISSIVO

Este capítulo consolida as diretrizes para replicação experimental rigorosa, fornecendo uma seção padronizada de 'Materiais e Métodos' pronta para inclusão direta em artigos científicos de alto impacto taxonômico, a Tabela Paramétrica Sintética integral do sistema e o Índice Remissivo Consolidado de todas as formulações e conceitos abordados.

6.1. Seção Padronizada 'Materiais e Métodos' para Publicações Científicas

O texto a seguir foi estruturado de acordo com as exigências metodológicas e terminológicas de periódicos internacionais de referência em entomologia e bioacústica sistemática (e.g., *Zootaxa*, *Systematic Entomology*, *Bioacoustics*, *Journal of Orthoptera Research*), permitindo sua transcrição direta para manuscritos acadêmicos:

*"As gravações bioacústicas de estridulação foram adquiridas em formato WAV não comprimido (taxas de amostragem de 44,1 kHz a 96,0 kHz; quantização de 16 ou 24 bits) e analisadas através da plataforma metrológica **Crinômetro (v4.5.0)**. Para gravações multicanal, o sistema selecionou automaticamente o canal com maior potência eficaz (RMS), prevenindo distorções por interferência destrutiva espacial (comb filtering). O sinal foi centralizado para remoção de offset DC e submetido a um filtro passa-altas Butterworth de 2ª ordem com corte em 50 Hz para atenuação de ruídos de vento aerodinâmico, seguido de filtro passa-faixa Butterworth de 4ª ordem sintonizado na banda estridulatória do táxon (3200–6000 Hz). Ambos os filtros foram computados em Seções de Segunda Ordem (SOS / Biquads) bidirecionais de fase zero (sosfiltfilt; ordem efetiva 8; atenuação de -48 dB/oitava; atraso de grupo identicamente nulo), garantindo a preservação absoluta da geometria temporal dos pulsos.*

A análise tempo-frequência foi calculada via Transformada de Fourier de Tempo Curto (STFT) com janelas de Hann de 1024 amostras e sobreposição temporal de 75% (hop size de 256 amostras; resolução temporal $\Delta t \approx 5,3$ a 5,8 ms; resolução frequencial $\Delta f \approx 43$ a 47 Hz). A frequência dominante instantânea foi refinada por interpolação parabólica contínua sub-bin de Smith-Serra. A Densidade Espectral de Potência (PSD) média foi estimada pelo método de Welch utilizando janelas de 50 ms sobrepostas em 50%, impondo uma resolução frequencial invariante de exatamente 20,0 Hz.

A envoltória instantânea de modulação de amplitude (AM) foi extraída através do módulo da Transformada de Hilbert, expandida para o próximo inteiro 5-suave (next_fast_len), normalizada pelo percentil 99.85 da distribuição de amplitude e suavizada por convolução com janela de Hann de 2,0 ms. Os pulsos estridulatórios foram detectados por limiar adaptativo baseado no 25º percentil do envelope (P25 × 2.5) e validados morfologicamente pela largura a meia altura (-6 dB / FWHM). O agrupamento sintático em chilreios (chirps/echemas) operou através de um pipeline hierárquico em 5 camadas, compreendendo janelamento por Intervalo Inter-Pulso (IPI), fusão de colisões sub-refratárias (< 0.80 gap_min), coerência de amplitude fisiológica, portão refratário inter-chilreio (ICI Gate adaptativo a 70% da mediana) e filtro de coerência rítmica linear O(N) que expurga transientes que violem a cadência neuromotora por mais de ±30%.

A estabilidade do ritmo do Gerador Central de Padrões (CPG) ao longo da emissão foi inferida por regressão linear ordinária (OLS) dos intervalos inter-chilreios contínuos e testada estatisticamente via distribuição t de Student ($\alpha = 0,05$), após a segregação estrita de pausas fisiológicas espontâneas (ICIs > 3× a mediana). Todas as grandezas biométricas reportadas representam medições físicas diretas (frequências em Hz, intervalos e durações em ms), estando a análise totalmente isenta de estimativas térmicas empíricas derivadas da Lei de Dolbear, resguardando a independência metrológica indispensável para a diagnose e descrição formal de novas espécies."

6.2. Tabela Paramétrica Sintética Completa do Crinômetro v4.5.0

A Tabela 6.1 consolida os 16 parâmetros internos da plataforma, detalhando seus valores padrão, faixas operacionais permitidas e justificativas metrológicas:

Parâmetro Interno	Valor Padrão	Faixa Operacional	Unidade	Descrição Metrológica
b1_min	3200.0	20.0 a (Nyq - 100)	Hz	Frequência de corte inferior do filtro passa-faixa estridulatório.
b1_max	6000.0	120.0 a (Nyq - 20)	Hz	Frequência de corte superior do filtro passa-faixa (clampeada em Nyquist).
freq_tolerance_hz	700.0	100.0 a 2000.0	Hz	Semilargura da janela de tolerância espectral em torno de fc.
amp_min	0.08	0.01 a 0.50	Norm [0-1]	Limiar primário de amplitude relativa da envoltória normalizada.

Parâmetro Interno	Valor Padrão	Faixa Operacional	Unidade	Descrição Metrológica
noise_floor	1.00	0.20 a 3.00	Fator	Escalador de sensibilidade de ruído multiplicado pelo 25º percentil.
smooth_window_ms	2.0	0.5 a 10.0	ms	Largura temporal do kernel de Hann para suavização da envoltória.
prominence	0.015	0.005 a 0.10	Norm [0-1]	Proeminência topográfica mínima para validação de cristas de sílabas.
gap_min	25.0	5.0 a 80.0	ms	Intervalo inter-pulso (IPI) mínimo para continuidade intra-chilreio.
gap_max	35.0	15.0 a 150.0	ms	Intervalo inter-pulso (IPI) máximo admitido dentro do mesmo chilreio.
dur_min	14.0	2.0 a 40.0	ms	Duração temporal mínima admitida para uma sílaba a -6 dB (FWHM).
dur_max	80.0	30.0 a 250.0	ms	Duração temporal máxima admitida para uma sílaba a -6 dB.
min_p	2	1 a 20	pulsos	Número mínimo de pulsos validados para consolidação de um chilreio.
max_p	8	2 a 50	pulsos	Número máximo de pulsos admitidos por chilreio sintático.
ici_min	200.0	50.0 a 1000.0	ms	Intervalo inter-chilreio de referência para o portão refratário adaptativo.
focal_sensitivity	0.60	0.30 a 0.90	Norm [0-1]	Sensibilidade do classificador híbrido ML para resgate contextual.
psd_window_ms	50.0	20.0 a 200.0	ms	Janela de Welch para fixação da resolução espectral invariante em 20 Hz.

6.3. Índice Remissivo Consolidado de Fórmulas, Classes, Métodos e Glossário ([G-01] a [G-30])

A Tabela 6.2 apresenta o mapeamento cruzado entre os conceitos teóricos, as formulações matemáticas, os arquivos-fonte da versão 4.5.0 e os códigos remissivos do glossário:

Cód.	Conceito / Símbolo	Expressão / Função Principal	Módulo / Classe no Código v4.5.0	Seção
G-01	SSOT	SessionContext.state_changed	core/session_context.py (SessionContext)	§1.1, §1.2
G-02	Batch Atômico	with ctx.batch_update():	core/session_context.py (batch_update)	§1.2
G-03	Abort Cooperativo	GenericWorker._abort_event.set()	core/worker.py (GenericWorker)	§1.3
G-04	Deadline Monotônico	deadline = time.monotonic() + T	core/worker.py (WorkerRegistry)	§1.3
G-05	Atomic Replace	os.replace(temp_path, path)	utils/persistence.py (atomic_write)	§1.4
G-06	Seleção RMS Estéreo	c* = argmax_c √(mean(X_c²))	core/analyzer.py (CricketAnalyzer.analyze)	§2.1
G-07	Filtro SOS Anti-Vento	butter(2, 50/nyq, 'high', output='sos')	core/analyzer.py (CricketAnalyzer.analyze)	§2.1
G-08	Passa-Faixa Zero-Phase	sosfiltfilt(sos_b1, data)	core/analyzer.py (CricketAnalyzer.analyze)	§2.2
G-09	STFT Gabor-Heisenberg	spectrogram(window='hann', N=1024)	core/analyzer.py (CricketAnalyzer.analyze)	§2.3
G-10	Refinamento Sub-Bin	p = 0.5 (α-γ)/(α-2β+γ)	core/analyzer.py (CricketAnalyzer.analyze)	§2.4
G-11	PSD Welch Invariante	welch(nperseg=sr*0.05)	core/analyzer.py (compute_psd)	§2.5
G-12	Portadora Modal fc	Média ponderada no bin modal	core/analyzer.py (CricketAnalyzer.analyze)	§2.5, §3.4
G-13	Sintaxe CPG	Pulsos -> IPI -> Chilreios -> ICI	core/analyzer.py (regroup_chirps)	§3.1
G-14	Envoltória de Hilbert	E = √(x² + H{x}²)	core/analyzer.py (CricketAnalyzer.analyze)	§3.2
G-15	Duração FWHM (-6 dB)	d = t_right,0.5 - t_left,0.5	core/analyzer.py & core/learner.py	§3.3
G-16	Duty Cycle	DC = (d_p / IPI) × 100%	utils/report_generator.py	§3.3, §3.8
G-17	Coerência Rítmica O(N)	Poda periférica por mediana de IPI	core/analyzer.py (regroup_chirps)	§3.5
G-18	Portão ICI (ICI Gate)	Fusão se ICI < 0.70 med(ICI)	core/analyzer.py (regroup_chirps)	§3.5, §3.6
G-19	Vetor 20 Atributos	f(p_i) em R²⁰ normalizado	core/learner.py (extract_features_for_peak)	§4.1, §4.2
G-20	GMM Bimodal	EM com covariâncias completas	core/learner.py (PulseLearner)	§4.3

Cód.	Conceito / Símbolo	Expressão / Função Principal	Módulo / Classe no Código v4.5.0	Seção
G-21	HistGradientBoosting	Loss log-loss com L2 = 2.0	core/learner.py (PulseLearner)	§4.4
G-22	Contextual Gating	Resgate por IPI pareado	core/learner.py (filter_peaks)	§4.6
G-23	Auditoria skops	Validação AST de tipos seguros	core/learner.py (load_persisted_training)	§4.8
G-24	Locks Desacoplados	_model_lock isolado de CONFIG_LOCK	core/learner.py (save_to_config)	§4.8
G-25	Min-Max LoD	Redução de N para 2B vértices	core/engines.py (HighPerfLineEngine)	§5.1
G-26	Engines Vetorizadas	reshape(n_buckets, bucket_size)	core/engines.py (HighPerfLineEngine)	§5.2
G-27	Raster Blitting	copy_from_bbox / restore_region	ui/controllers/navigation_controller.py	§5.3
G-28	Poda Hover O(log M)	np.searchsorted em ±30 pixels	ui/controllers/navigation_controller.py	§5.3, §5.4
G-29	Debounce Espectral	QTimer 25 ms na portadora	ui/controllers/pipeline_orchestrator.py	§5.5
G-30	FigureFactory Pura	FigureCanvasAgg sem pyplot global	ui/figure_factory.py (ExportConfig)	§5.6